

Technická univerzita v Košiciach
Fakulta elektrotechniky a informatiky

Zvýšenie bezpečnosti vybraných
šifrovacích algoritmov v prístupových
systémoch

Diplomová práca

Technická univerzita v Košiciach
Fakulta elektrotechniky a informatiky

**Zvýšenie bezpečnosti vybraných
šifrovacích algoritmov v prístupových
systémoch**

Diplomová práca

Študijný program: Informatika
Študijný odbor: Informatika
Školiace pracovisko: Katedra počítačov a informatiky (KPI)
Školiteľ: doc. Ing. Eva Chovancová, PhD.
Konzultant:

Košice 2026

Oleksandr Mykhailyshyn

Abstrakt v SJ

Diplomová práca sa zaoberá návrhom, implementáciou a experimentálnym overením bezpečnostných mechanizmov pre systém kontroly fyzického prístupu na báze RFID/NFC. Ako srovnávacia základňa slúži štandardné nasadenie AEAD podľa RFC 8439; práca navrhuje modifikácie spôsobu jeho nasadenia zamerané na deterministickú konštrukciu nonce a runtime verifikáciu s anomálnou detekciou. Prototyp pozostáva z hardvérovej čítačky (ESP32), serverovej aplikácie v jazyku C++20 a lokálneho úložiska SQLite, a implementuje viacvrstvovú kryptografickú ochranu zahŕňajúcu autentifikáciu požiadaviek, šifrovanie s väzbou hlavičky, deriváciu kľúčov, anti-replay mechanizmus a tamper-evident auditný log. Experimentálna analýza porovnáva navrhované konfiguračné profily vo viacerých útočných scenároch a posudzuje ich prínos oproti štandardnému nasadeniu. Výsledky potvrdzujú, že kombinácia deterministického nonce a runtime detekcie poskytuje merateľné zlepšenie odolnosti pri zanedbateľnom dopade na výkon.

Kľúčové slová

kontrola prístupu, RFID, NFC, kryptografia, AEAD, ChaCha20-Poly1305, XChaCha20-Poly1305, anomálna detekcia, HMAC, HKDF, anti-replay, audit log

Abstrakt v AJ

The thesis presents the design, implementation, and experimental evaluation of security mechanisms for an RFID/NFC physical access control system. The baseline is the standard AEAD deployment per RFC 8439; the work proposes modifications to the deployment mode focused on deterministic nonce construction and runtime verification with anomaly detection. The prototype consists of a hardware reader (ESP32), a C++20 server application, and a local SQLite storage, and implements multi-layered cryptographic protection including request authentication, encryption with header binding, key derivation, an anti-replay mechanism, and a tamper-evident audit log. The experimental analysis compares the proposed configuration profiles across multiple attack scenarios and assesses their contribution relative to the standard deployment. The results confirm that combining a deterministic nonce with runtime detection yields a measurable improvement in resilience at a negligible impact on performance.

Klíčové slová v AJ

access control, RFID, NFC, cryptography, AEAD, ChaCha20-Poly1305, XChaCha20-Poly1305, anomaly detection, HMAC, HKDF, anti-replay, audit log

Zadanie práce

Namiesto tejto strany vložte naskenované zadanie úlohy. Odporúčame skenovať s rozlíšením 200 až 300 dpi, čierno-bielo! V jednej vytlačenej ZP musí byť vložený originál zadávacieho listu!

Čestné vyhlásenie

Vyhlasujem, že som diplomovú prácu vypracoval(a) samostatne s použitím uvedenej odbornej literatúry.

Košice 24.04.2026

.....

Vlastnoručný podpis

Poďakovanie

Ďakujem vedúcemu diplomovej práce za odborné vedenie, cenné rady a trpezlivú podporu počas celého procesu vypracovania tejto práce.

Predhovor

Diplomová práca sa zaoberá návrhom a experimentálnym overením bezpečnostných mechanizmov pre systém kontroly prístupu. Práca vznikla na Katedre počítačov a informatiky Fakulty elektrotechniky a informatiky Technickej univerzity v Košiciach. Motiváciou bolo zvýšenie dôveryhodnosti komunikácie medzi čítačkou a serverom v prostredí, kde fyzická aj informačná bezpečnosť konvergujú.

Obsah

Úvod	17
1 Formulácia úlohy	19
1.1 Úloha 1: Analyzovať aktuálny stav moderných prístupových systémov	19
1.2 Úloha 2: Analyzovať problematiku bezpečnosti šifrovacích algoritmov	19
1.3 Úloha 3: Navrhnuť model prístupového systému	20
1.4 Úloha 4: Vyhodnotiť účinnosť algoritmov a navrhnuť modifikácie . . .	20
1.5 Úloha 5: Vypracovať dokumentáciu podľa štandardov KPI	21
1.6 Podmienky a obmedzenia riešenia	21
2 Analýza	22
2.1 Analýza aktuálnych riešení a srovnávací základňa	22
2.1.1 Existujúce riešenia v oblasti prístupových systémov	22
2.1.2 RFC 8439 ako srovnávacia základňa	23
2.2 Bezpečnostné východiská a model hrozieb	24
2.2.1 Bezpečnostné požiadavky ako merateľné tvrdenia	25
2.2.2 Model hrozieb STRIDE	25
2.2.3 Špecifiká RFID/NFC v prístupových systémoch	25
2.3 Systémy kontroly prístupu a prístupové modely	26
2.3.1 Fyzická vs. logická kontrola prístupu	26
2.3.2 Modely kontroly prístupu	26
2.3.3 Princíp najmenej oprávnení	27
2.4 Kryptografické základy	27
2.4.1 HMAC a hashovacie funkcie	27
2.4.2 Symetrické šifrovanie a AEAD	28
2.4.3 Nonce stratégie a ich bezpečnostné modely	30
2.4.4 HKDF a domain separation	31
2.5 Zásady návrhu bezpečných protokolov	31

2.5.1	Fail-closed správanie	31
2.5.2	Jednoznačná serializácia a validácia vstupu	31
2.5.3	Mapovanie bezpečnostných cieľov na mechanizmy	32
2.5.4	Kompromisy medzi bezpečnosťou a použiteľnosťou	32
2.6	Správa kryptografických kľúčov	33
2.6.1	Oddelenie kľúčov podľa účelu	33
2.6.2	Verzovanie a rotácia kľúčov	33
2.6.3	Ukladanie kľúčov a provisioning	33
2.6.4	Kompromitácia a postup obnovy	34
2.7	Ochrana súkromia identifikátorov	34
2.8	Tamper-evident audit log	34
2.8.1	Hash chain s HMAC	34
2.8.2	Audit anchor a ochrana proti truncation	35
2.8.3	Forward integrity	35
2.8.4	Verifikácia	35
2.9	Bezpečnosť vstavaného zariadenia	35
2.9.1	Ukladanie tajomstiev na zariadení	35
2.9.2	Monotónne počítadlá a trvácnosť stavu	36
2.9.3	Fyzické útoky a Wi-Fi konfigurácia	36
2.10	Bezpečnosť transportnej vrstvy	36
2.10.1	Prečo nestačí iba TLS	36
2.10.2	Praktické nasadenie TLS v IoT	37
2.11	Súvisiace prístupy a postavenie navrhovaného riešenia	37
2.11.1	Komunikačné protokoly: Wiegand a OSDP	37
2.11.2	Postavenie navrhovaného riešenia	38
2.12	Zhrnutie teoretických východísk	38
2.12.1	Predpoklady a zvyškové riziko	38

3 Realizácia a experimentálne vyhodnotenie

40

3.1	Praktická implementácia navrhnutého systému	40
3.1.1	Architektúra implementácie	42
3.1.2	Modulárna štruktúra a externé závislosti	43
3.1.3	AEAD šifrovanie interného frame	44
3.1.4	Derivácia kľúčov a domain separation	47
3.1.5	Anti-replay mechanizmus	49
3.1.6	Detektor protokolových anomálií	50
3.1.7	Ďalšie bezpečnostné mechanizmy	52
3.2	Verifikácia kvality kódu pred experimentmi	53
3.3	Experimentálna analýza bezpečnostných profilov	53
3.3.1	Matica konfiguračných profilov	55
3.3.2	Architektúra experimentálneho harnessu	56
3.3.3	Popis útokových scenárov	59
3.3.4	Metodika zberu a spracovania dát	62
3.3.5	Výsledky: Úspešnosť útokov	64
3.3.6	Výsledky: Baseline korektnosť	66
3.3.7	Výsledky: Latencia a priepustnosť	66
3.3.8	Diskusia a interpretácia výsledkov	67
3.3.9	Porovnanie s minimalistickými implementáciami	68
3.3.10	Zhrnutie experimentov	69
4	Záver (zhodnotenie riešenia)	71
	Zoznam použitej literatúry	73
	Zoznam príloh	78
	Príloha A	79
	Príloha B	80

FEI	KPI
Príloha C	81
Príloha D	82

Zoznam obrázkov

3-1	Derivácia kľúčov z master secret cez HKDF. Každá vetva má vlastný <code>info</code> reťazec (domain separation).	48
3-2	Úspešnosť útokov podľa scenára a profilu (priemer cez 5 behov). Zelené bunky: útok zablokovaný (0%). Červené bunky: útok úspešný.	64
3-3	Priepustnosť pri $N = 50,000$ frame (priemer $\pm$ std cez 5 behov) a percentily latencie.	66

Zoznam tabuliek

1–1	Prepojenie úloh zadania s kapitolami práce	21
2–1	Porovnanie ChaCha20-Poly1305 a AES-GCM	30
3–1	Hlavné vrstvy implementácie a ich úloha v prototype	43
3–2	Prehľad HKDF derivácie kľúčov z master secret	48
3–3	Typy anomálií sledovaných detektorom <code>ProtocolAnomalyDetector</code> .	51
3–4	Konfigurácia <code>ProtocolAnomalyDetector</code> a reakcia na anomálie . . .	52
3–5	Mapovanie STRIDE kategórií hrozieb na experimentálne scenáre . . .	54
3–6	Matica konfiguračných profilov (algoritmus $\times$ režim nonce)	55
3–7	Úspešnosť útoku a reakcia detektora podľa scenára a nonce stratégie .	64
3–8	Priepustnosť a latencia pri $N=50,000$ frame	67
3–9	Odolnosť voči útočným scenárom podľa úrovne implementácie	69
4–1	Súhrn hlavných experimentálnych zistení	72

Slovník termínov

AEAD	Authenticated Encryption with Associated Data
ABAC	Attribute-Based Access Control
AAA	Authentication, Authorization, Accounting
ARX	Add-Rotate-XOR
CIA	Confidentiality, Integrity, Availability
DAC	Discretionary Access Control
DoS	Denial of Service
ESP32	mikrokontrolér Espressif Systems ESP32
FEI	Fakulta elektrotechniky a informatiky
FIPS	Federal Information Processing Standard
GDPR	General Data Protection Regulation
HKDF	HMAC-based Key Derivation Function
HMAC	Hash-based Message Authentication Code
HSM	Hardware Security Module
IoT	Internet of Things
KPI	Katedra počítačov a informatiky
LSan	LeakSanitizer
MAC	Message Authentication Code
MITM	Man-in-the-Middle (útok)

mTLS	mutual Transport Layer Security
NFC	Near Field Communication
NIST	National Institute of Standards and Technology
NVS	Non-Volatile Storage
OSDP	Open Supervised Device Protocol
OTA	Over-The-Air (bezdrôtová aktualizácia)
OWASP	Open Web Application Security Project
PKI	Public Key Infrastructure
PSK	Pre-Shared Key
RBAC	Role-Based Access Control
RC522	čítačka RFID kariet (NXP Semiconductors)
RFC	Request for Comments
RFID	Radio Frequency Identification
SIMD	Single Instruction, Multiple Data
SQLite	odľahčená relačná databáza (serverless)
STRIDE	Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege
TLS	Transport Layer Security
TUKE	Technická univerzita v Košiciach
UBSan	UndefinedBehaviorSanitizer
UID	Unique Identifier

Úvod

Systémy kontroly fyzického prístupu patria medzi prvky infraštruktúry, pri ktorých sa fyzická a informačná bezpečnosť stretávajú v jednom bode. Hoci sa na prvý pohľad jedná o jednoduché rozhodnutie — otvoriť alebo neotvoriť dvere — v pozadí stojí identifikácia používateľa, vyhodnotenie oprávnení, kryptografická ochrana komunikácie a auditovateľnosť udalostí.

V súčasnosti sa v praxi stále bežne používajú jednoduchšie riešenia na báze prenosu surového UID karty bez kryptografickej ochrany, často v kombinácii s protokolom Wiegand, ktorý neposkytuje autentifikáciu ani ochranu proti replay útokom. Priemyselné systémy tieto nedostatky riešia proprietárnymi protokolmi, ktorých bezpečnostné vlastnosti nie sú verejne overiteľné. Otvorený štandard OSDP v2 pridáva šifrovanú komunikáciu, no nepokrýva pokročilé mechanizmy ako per-reader deriváciu kľúčov či runtime detekciu anomálií.

Cieľom tejto práce je navrhnúť a experimentálne overiť modifikácie štandardného nasadenia AEAD algoritmu ChaCha20-Poly1305 podľa RFC 8439, ktoré zvýšia odolnosť prístupového systému voči vybraným triedam útokov. Konkrétne ide o deterministickú konštrukciu nonce, runtime verifikáciu s anomálnou detekciou a karanténou zariadení, a per-reader deriváciu kľúčov pomocou HKDF. Ako metóda overenia slúži experimentálne porovnanie šiestich konfiguračných profilov v siedmich útokových a výkonnostných scenároch, kde štandardné nasadenie podľa RFC 8439 (náhodný nonce, bez detekcie) tvorí srovnávaciu základňu.

Práca vychádza z funkčného prototypu postaveného na platforme ESP32 s čítačkou RC522, serverovej aplikácii v jazyku C++20 a lokálnom úložisku SQLite. Architektúra oddeľuje spracovanie hardvérových požiadaviek od rozhodovacej logiky prostredníctvom HTTP rozhrania, čo umožňuje nezávislé testovanie bezpečnostných vlastností AEAD vrstvy. Práca nadväzuje na existujúce výskumy v oblasti nonce-misuse resistance a bezpečného nasadenia AEAD konštrukcií a dopĺňa ich o praktickú implementáciu a merateľné experimentálne výsledky v kontexte

embedded prístupového systému.

Štruktúra práce je nasledovná: kapitola 1 formuluje ciele a obmedzenia, kapitola 2 analyzuje existujúce riešenia a teoretické východiská, kapitola 3 opisuje implementáciu prototypu a experimentálnu analýzu a kapitola 4 zhŕňa výsledky a odporúčania pre ďalší vývoj.

1 Formulácia úlohy

Zadanie diplomovej práce definuje päť úloh. Táto kapitola rozvädza spôsob, akým sú jednotlivé úlohy riešené, a uvádza podmienky a obmedzenia riešenia. Prepojenie úloh s kapitolami práce sumarizuje tabuľka 1 – 1.

1.1 Úloha 1: Analyzovať aktuálny stav moderných prístupových systémov

Analýza aktuálneho stavu pokrýva existujúce riešenia od jednoduchých systémov na báze UID a protokolu Wiegand, cez otvorený štandard OSDP v2 s AES-128 Secure Channel, až po priemyselné systémy s proprietárnymi protokolmi. Pre každú kategóriu sa identifikujú bezpečnostné vlastnosti a nedostatky relevantné pre kontext tejto práce. Ako srovnávacía základňa pre experimentálne overenie slúži štandardné nasadenie AEAD podľa RFC 8439 (ChaCha20-Poly1305), ktoré definuje algoritmus a požiadavky na nonce, no nepredpisuje spôsob generovania nonce, deriváciu kľúčov pre viacero zariadení ani runtime detekciu anomálií.

1.2 Úloha 2: Analyzovať problematiku bezpečnosti šifrovacích algoritmov

Teoretická analýza sa zameriava na kryptografické primitíva priamo použité v prototyp: AEAD konštrukcie ChaCha20-Poly1305 a XChaCha20-Poly1305, ich bezpečnostné modely (IND-CPA, INT-CTXT, nonce-misuse resistance), HMAC-SHA-256 pre autentifikáciu, HKDF pre deriváciu kľúčov a konštrukciu tamper-evident auditného logu. Porovnanie s AES-GCM zdôvodňuje voľbu algoritmov pre embedded platformu bez hardvérovej akcelerácie. Analýza identifikuje tri stratégie generovania nonce (náhodný, deterministický HMAC, deterministický s runtime verifikáciou) ako hlavnú experimentálnu os.

1.3 Úloha 3: Navrhnuť model prístupového systému

Model je realizovaný ako funkčný prototyp: hardvérová čítačka (ESP32 s modulom RC522) komunikuje so serverovou aplikáciou v jazyku C++20 prostredníctvom HTTP požiadaviek autentifikovaných HMAC-SHA-256. Server implementuje viacvrstvovú ochranu: interný AEAD frame s AAD binding, HKDF per-reader deriváciu kľúčov, anti-replay sliding window, detektor protokolových anomálií s karanténou a tamper-evident audit log. Architektúra oddeľuje spracovanie požiadaviek od rozhodovacej logiky prostredníctvom HTTP rozhrania medzi dvoma nezávislými službami (frontend a DecisionService), čo umožňuje nezávislé testovanie bezpečnostných vlastností AEAD vrstvy. Šesť konfiguračných profilov (dva algoritmy krát tri nonce stratégie) vytvára experimentálnu maticu, kde všetky ostatné mechanizmy zostávajú konštantné.

1.4 Úloha 4: Vyhodnotiť účinnosť algoritmov a navrhnuť modifikácie

Vyhodnotenie prebieha experimentálne pomocou C++ harnessu, ktorý zdieľa produkčný kód a umožňuje reprodukovateľné merania. Sedem scenárov testuje odolnosť voči replay útoku, manipulácii hlavičky, nonce enforcement, cross-reader izolácii kľúčov, rollbacku sekvenčného čísla, verifikácii AEAD tagu a nonce tamper útoku na kanáli. Modifikácia oproti štandardnému nasadeniu RFC 8439 spočíva v deterministickej konštrukcii nonce (HMAC z kontextu hlavičky), runtime verifikácii nonce na serveri a automatickej karantény zariadení pri detekcii anomálie. Experimenty bežia v dvoch režimoch: in-process (čisté kryptografické metriky) a E2E cez HTTP endpoint (validácia celej spracovateľskej cesty).

1.5 Úloha 5: Vypracovať dokumentáciu podľa štandardov KPI

Práca je vypracovaná podľa šablóny a štandardov KPI FEI TUKE. Zdrojový kód, experimentálne dáta a analytický pipeline sú súčasťou priloženého média.

1.6 Podmienky a obmedzenia riešenia

Prototyp vedome zjednodušuje niektoré aspekty produkčného nasadenia. Karta pracuje v jednoduchom režime — UID vystupuje ako identifikátor, nie ako kryptografický dôkaz identity. Úložisko je postavené na SQLite, čo je praktické pre prototyp a experimenty, no neposkytuje produkčné škálovanie. Povinné TLS nie je v základnom režime nasadené; dôvernosť prenosu UID je riešená na aplikačnej úrovni. Tieto obmedzenia vytvárajú presný rámec pre interpretáciu výsledkov — ak sa ukáže zlepšenie odolnosti, ide o prínos navrhnutých modifikácií v rámci definovaného prototypu, nie o všeobecné tvrdenie o bezpečnosti prístupových systémov.

Tabuľka 1 – 1 Prepojenie úloh zadania s kapitolami práce

Úloha	Obsah	Kapitola
1	Analýza aktuálnych riešení a srovnávacia základňa	2
2	Kryptografické základy, nonce stratégie, HKDF	2
3	Návrh a implementácia prototypu	3
4	Experimentálna analýza (7 scenárov, 6 profilov)	3
5	Dokumentácia	celá práca

2 Analýza

Táto kapitola zhŕňa teoretické východiská potrebné pre návrh systému: existujúce riešenia kontroly prístupu, bezpečnostné požiadavky, modely oprávnení, kryptografické primitívy a zásady návrhu bezpečných protokolov. Cieľom je vymedziť rámec, v ktorom sa pohybuje praktická časť práce, a identifikovať aspekty, ktoré štandardné riešenia nepokrývajú dostatočne pre konkrétny scenár RFID/NFC prístupového systému.

2.1 Analýza aktuálnych riešení a srovnávací základňa

Pred návrhom vlastných modifikácií je potrebné zhodnotiť existujúce prístupy k bezpečnosti RFID/NFC systémov a vymedziť srovnávaciu základňu, voči ktorej bude navrhované riešenie experimentálne hodnotené. Nasledujúce podkapitoly mapujú spektrum reálne nasadených riešení a identifikujú štandard, ktorý slúži ako referenčný bod.

2.1.1 Existujúce riešenia v oblasti prístupových systémov

V praxi existujú rôzne prístupy k bezpečnosti prístupových systémov. Najjednoduchšie riešenia na báze ESP32 a RC522 (napr. ESP-RFID[37]) prenášajú surové UID cez HTTP bez podpisu, bez anti-replay mechanizmu a bez auditu. Legacy protokol Wiegand[9] neposkytuje šifrovanie ani autentifikáciu — je jednosmerný prenos identifikátora bez akejkoľvek kryptografickej väzby. Na opačnom konci spektra stoja priemyselné systémy s proprietárnymi protokolmi[13, 2] a otvorený štandard OSDP v2[33] s AES-128 Secure Channel. Tieto riešenia poskytujú rôznu úroveň ochrany. Prototyp tejto práce sa zameriava na konkrétne mechanizmy, ktoré v jednoduchších riešeniach typicky chýbajú a v priemyselných nie sú verejne dokumentované na úrovni umožňujúcej nezávislé experimentálne overenie: per-reader deriváciu kľúčov (HKDF), runtime anomálnu detekciu s karanténou a tamper-evident audit.

2.1.2 RFC 8439 ako srovnávacia základňa

Hlavnou srovnávacou základňou tejto práce je štandardné nasadenie AEAD podľa RFC 8439[26]. Tento štandard definuje algoritmus ChaCha20-Poly1305, formát AEAD operácie, použitie AAD (Additional Authenticated Data) a požiadavky na nonce: nonce sa nesmie opakovať pod rovnakým kľúčom[23]. Štandard však **nepredpisuje**, akým spôsobom má byť nonce generovaný (náhodne vs. deterministicky), neobsahuje odporúčania pre deriváciu kľúčov pre viacero zariadení, nedefinuje anti-replay mechanizmus na aplikačnej úrovni a neobsahuje runtime detekciu zneužitia nonce.

Práve tieto aspekty — spôsob generovania nonce, derivácia kľúčov a runtime detekcia anomálií — sú predmetom modifikácií navrhnutých v praktickej časti tejto práce. Výskum v oblasti nonce-misuse resistance[29, 4] ukazuje, že deterministická konštrukcia nonce znižuje závislosť na kvalite generátora náhodných čísiel a umožňuje serverovú verifikáciu zhody nonce.

Pre aspekty, ktoré RFC 8439 nepokrýva, slúžia ako ďalšie orientéry:

- **Derivácia kľúčov:** HKDF podľa RFC 5869[19], s per-reader salt pre kryptografickú izoláciu zariadení.
- **Anti-replay:** Sliding window mechanizmus, bežný v protokoloch ako IPsec a DTLS[23].
- **Anomálna detekcia:** Princípy z NIST SP 800-94[31] aplikované na protokolovú úroveň.
- **Tamper-evident audit:** Konštrukcia podľa Schneier a Kelsey[32].
- **Rozšírený nonce (XChaCha20):** Libsodium/IETF draft[1, 21] pre 192-bitový nonce.

2.2 Bezpečnostné východiská a model hrozieb

Bezpečnostný návrh prístupového systému vychádza z CIA triády[15] — dôvernosti, integrity a dostupnosti — a rozširuje ju o ďalšie ciele relevantné pre komunikáciu medzi zariadeniami a serverom:

Dôvernosť vyžaduje, aby útočník nemohol získať citlivé údaje (identifikátory kariet, prístupové tokeny) ani zo zachytenej komunikácie, ani z databázy. Typickým riešením je šifrovanie komunikácie a pseudonymizácia identifikátorov v úložisku.

Integrita vyžaduje detekciu akejkoľvek neoprávnenej zmeny — či už v prenášanej správe, v metadátach autorizačného kontextu, alebo v auditnej stope. Kryptografické mechanizmy (MAC, AEAD s AAD, hash chain) zabezpečujú, že zmena je detekovaná bez ohľadu na to, kde k nej došlo.

Dostupnosť v kontexte prístupových systémov znamená, že systém musí spoľahlivo reagovať na oprávnené požiadavky a zároveň odolávať pokusom o zahltenie. Princíp fail-closed zabezpečuje, že pri neistote systém požiadavku zamietne namiesto povolenia prístupu.

Autentickosť vyžaduje overenie pôvodu každej požiadavky. V systémoch s hardvérovými čítačkami je potrebné overiť, že požiadavka skutočne pochádza z registrovaného zariadenia, nie od útočníka. HMAC s tajným kľúčom je štandardným riešením na aplikačnej úrovni[24].

Odolnosť voči replay znamená, že zachytená platná požiadavka nemôže byť úspešne zopakovaná. Riešením sú monotónne sekvenčné čísla v kombinácii so sliding window mechanizmom na strane servera.

Auditovateľnosť vyžaduje, aby bolo možné po incidente spätne preukázať, čo sa stalo, a odhaliť prípadnú manipuláciu s históriou udalostí. Tamper-evident logovanie s kryptografickým reťazením záznamov[32] zabezpečuje detekciu modifikácie, vloženia aj vymazania.

2.2.1 Bezpečnostné požiadavky ako merateľné tvrdenia

Pri návrhu bezpečnostného riešenia je užitočné formulovať požiadavky ako overiteľné tvrdenia. Napríklad:

- *Systém odmietne opakovanie tej istej požiadavky aj v prípade, že bola zachytená na sieti.*
- *Únik databázy nesmie priamo odhaliť surové identifikátory kariet.*
- *Po manipulácii s auditným záznamom je možné určiť prvé miesto nekonzistencie.*
- *Frame zašifrovaný kľúčom jednej čítačky nemôže byť úspešne overený v kontexte inej čítačky.*

Takto formulované požiadavky sa dajú priamo mapovať na konkrétne mechanizmy (anti-replay, HMAC hashovanie UID, hash chain, HKDF izolácia) a poskytujú jasné kritériá, čo sa považuje za „zvýšenie bezpečnosti“.

2.2.2 Model hrozieb STRIDE

Ako orientačný rámec slúži model STRIDE[12]: Spoofing (podvrhnutie požiadavky — riešené HMAC), Tampering (zmena dát alebo logov — riešené AEAD a audit hash chain), Repudiation (popieranie udalostí — riešené auditom), Information disclosure (únik identifikátorov — riešené hashovaním UID v DB), Denial of service (čiastočne riešené limitmi a validáciou). Tieto kategórie slúžia ako východisko pre identifikáciu relevantných útočných scenárov.

2.2.3 Špecifiká RFID/NFC v prístupových systémoch

RFID/NFC systémy čelia špecifickým hrozbám: skimming (neoprávnené čítanie UID), klonovanie kariet, relay útoky a manipulácia komunikácie medzi čítačkou a serverom[10, 36]. V jednoduchších systémoch UID vystupuje ako identifikátor, nie ako kryptografický dôkaz identity. Toto riziko je možné kompenzovať

silnejšou ochranou na komunikačnej a serverovej vrstve: autentifikáciou zariadenia, šifrovaním a integritou interných správ, anti-replay mechanizmom a auditovateľnou históriou.

Na komunikačnej vrstve sú relevantné útoky MITM (zachytenie a úprava požiadaviek), replay (opakovanie platných požiadaviek) a enumerácia (odvodenie existujúcich identifikátorov z chybových odpovedí). Na serverovej strane hrozí neautorizovaný prístup k databáze, manipulácia s auditnou stopou a zneužitie administrátorských oprávnení[31, 15].

2.3 Systémy kontroly prístupu a prístupové modely

Bezpečný návrh prístupového systému vyžaduje porozumenie tomu, aký typ kontroly systém vykonáva a aký model oprávnení používa. Táto podkapitola zhŕňa základné pojmy a modely relevantné pre scenár RFID/NFC kontroly fyzického prístupu.

2.3.1 Fyzická vs. logická kontrola prístupu

Fyzická kontrola prístupu rieši otázku, kto smie vstúpiť na konkrétne miesto. Logická kontrola prístupu sa zaoberá oprávneniami v informačných systémoch. V moderných systémoch sa tieto oblasti prekrývajú: fyzická čítačka komunikuje so serverom, ktorý overuje oprávnenia podľa logických pravidiel. Moderné prístupové systémy sú na tomto priesečníku — hardvérová čítačka je fyzický vstupný bod, no rozhodovanie a audit sú plne digitálne.

2.3.2 Modely kontroly prístupu

V literatúre existujú štyri základné modely:

- **DAC** (Discretionary Access Control): vlastník zdroja rozhoduje o oprávneniach.
- **MAC** (Mandatory Access Control): oprávnenia vyplývajú zo systémových pravidiel a klasifikácie.

- **RBAC** (Role-Based Access Control): oprávnenia sú naviazané na roly[30].
- **ABAC** (Attribute-Based Access Control): rozhodnutia na základe atribútov subjektu, objektu a kontextu.

Pre fyzické prístupové systémy je najčastejšie používaný **RBAC**: karte je priradená rola, dveriam množina povolených rolí a čítačke množina dverí. Prístup je povolený, ak rola karty patrí do priesečníku povolených rolí pre dané dvere cez danú čítačku. Tento model je dostatočne expresívny pre väčšinu fyzických prístupových scenárov a priamo mapovateľný na relačnú databázovú štruktúru[16].

2.3.3 Princíp najmenej oprávnení

Princíp najmenej oprávnení[30] požaduje, aby subjekt mal iba tie oprávnenia, ktoré sú nevyhnutné pre jeho úlohu. V prístupových systémoch sa tento princíp realizuje oddelením oprávnení: hardvérové zariadenie sa preukazuje iným typom poverenia než administrátor, každá čítačka má prístup iba k dverám, ku ktorým je explicitne priradená, a kompromitácia jedného zariadenia neodhaľuje oprávnenia ostatných.

2.4 Kryptografické základy

Kryptografické mechanizmy použité v prototype vychádzajú z ustálených primitív popísaných v tejto podkapitole: autentifikačné kódy správ, symetrické šifrovanie s overením integrity, stratégie generovania nonce a odvodzovanie kľúčov. Pre každú skupinu sú zhrnuté vlastnosti relevantné pre návrh riešenia.

2.4.1 HMAC a hashovacie funkcie

MAC s využitím tajného kľúča zabezpečuje integritu a autentickosť správy. HMAC je konkrétna konštrukcia MAC nad hashovacou funkciou, štandardizovaná vo FIPS 198-1[24, 20]. V prístupových systémoch sa HMAC typicky používa pre autentifikáciu požiadaviek od zariadení, pričom sa počíta nad jednoznačnou

(kanonickou) reprezentáciou správy a porovnáva konštantným časom. Kľúče pre rôzne účely sa oddeľujú pomocou HKDF[19].

2.4.2 Symetrické šifrovanie a AEAD

Symetrické šifrovanie je vhodné pre embedded zariadenia kvôli výkonu a jednoduchosti správy kľúčov (na rozdiel od asymetrickej kryptografie nevyžaduje PKI). Samotné šifrovanie však nerieši integritu, preto sa v moderných protokoloch používa AEAD (Authenticated Encryption with Associated Data), ktoré spája dôvernosť a integritu do jednej operácie.

ChaCha20 a Poly1305 ChaCha20 je prúdová šifra navrhnutá D. J. Bernsteinom[5]. Poly1305 je jednorázový MAC. Ich kombinácia je štandardizovaná v RFC 8439[26, 23].

Stav ChaCha20 je matica 4×4 32-bitových slov. Základnou operáciou je quarter round $QR(a, b, c, d)$ — ARX konštrukcia (Add-Rotate-XOR):

$$a += b; \quad d \oplus = a; \quad d \lll 16$$

$$c += d; \quad b \oplus = c; \quad b \lll 12$$

$$a += b; \quad d \oplus = a; \quad d \lll 8$$

$$c += d; \quad b \oplus = c; \quad b \lll 7$$

Dvadsať kôl (10 stĺpcových + 10 diagonálnych) transformuje stav na keystream blok, ktorý sa XOR-uje s plaintext-om. Všetky operácie sú identicky časované (žiadne tabuľkové vyhľadávania), čo eliminuje timing side-channel útoky[5].

Poly1305 je autentifikátor v poli $\mathbb{F}_{2^{130}-5}$:

$$\text{tag} = \left((m_1 r^n + m_2 r^{n-1} + \dots + m_n r) \bmod (2^{130} - 5) + s \right) \bmod 2^{128} \quad (2.1)$$

kde r a s sú odvodené z prvého keystream bloku ChaCha20 (counter = 0)[26].

XChaCha20-Poly1305 a nonce manažment AEAD konštrukcie sú citlivé na zneužitie nonce. XChaCha20-Poly1305 rozširuje nonce na 192 bitov, čo znižuje riziko kolízie pri náhodnom generovaní[1, 21]. ChaCha20-Poly1305 IETF (RFC 8439) používa 96-bitový nonce s limitom $\sim 2^{48}$ správ. Obe varianty sú vhodné pre prístupové systémy, pričom nonce môže byť generovaný náhodne alebo deterministicky pomocou HMAC.

Associated Data (AAD) AEAD podporuje Associated Authenticated Data: dáta, ktoré sa nešifrujú, ale ich integrita je kryptograficky chránená. V prístupových protokoloch sa AAD typicky používa pre hlavičku správy — metadáta ako identifikátor zariadenia, cieľové dvere alebo sekvenčné číslo sú kryptograficky viazané na šifrovaný obsah. Zmena akéhokoľvek z týchto polí spôsobí zlyhanie overenia tagu.

Porovnanie s AES-GCM AES-GCM je dominantnou AEAD konštrukciou v TLS 1.3 a podnikových systémoch. Tabuľka 2–1 porovnáva obe konštrukcie z hľadiska vlastností relevantných pre embedded nasadenie. Pre platformy bez hardvérovej akcelerácie AES (napr. ESP32[7]) je ChaCha20-Poly1305 konzistentne bezpečná voľba vďaka konštantnočasovej ARX konštrukcii[5].

Tabuľka 2 – 1 Porovnanie ChaCha20-Poly1305 a AES-GCM

Vlastnosť	ChaCha20-Poly1305	AES-GCM
Veľkosť nonce	96b (IETF) / 192b (XChaCha20)	96b
Nonce reuse	Únik XOR plaintextov (two-time pad)	Kritické: extrahovateľný GHASH kľúč[23]
Softvér bez HW	Konštantný čas (ARX)[5]	Timing side-channel bez AES-NI
HW akcelerácia	AVX2/SSSE3 (SIMD)	AES-NI (Intel/ARM)
Vhodnosť pre IoT	Vysoká	Iba s HW akcelerátorom
Štandardizácia	RFC 8439, TLS 1.3	RFC 5116, TLS 1.3

2.4.3 Nonce stratégie a ich bezpečnostné modely

V literatúre a praxi existujú tri základné stratégie generovania nonce pre AEAD:

- **Náhodný nonce** — závisí na kvalite RNG; pri krátkom nonce (96 b) hrozí kolízia pri vysokom počte správ.
- **Deterministický nonce** — $\text{HMAC}(K_{\text{nonce}}, \text{context})$; eliminuje závislosť na RNG.
- **Deterministický s runtime verifikáciou** — rovnaký HMAC nonce, ale server overuje zhodu s očakávanou hodnotou; pri odchýlke zakaranténuje zariadenie.

Formálne sa AEAD bezpečnosť definuje kombináciou IND-CPA (dôvernosť) a INT-CTXT (integrita)[23]. Štandardný model (nAE) predpokladá unikátne nonce. Deterministická konštrukcia túto podmienku zabezpečuje bez závislosti na RNG; runtime verifikácia navyše aktívne vynucuje nonce politiku na serveri. Ide o praktický kompromis medzi nAE a plnou nonce-misuse resistance[29, 4].

2.4.4 HKDF a domain separation

Z jedného master secret sa pomocou HKDF[19] odvádzajú podkľúče pre rôzne účely s rôznymi info parametrami (domain separation). Typické derivácie v prístupovom systéme zahŕňajú: kľúč pre AEAD šifrovanie, kľúč pre generovanie nonce, kľúč pre auditný hash chain a pepper pre pseudonymizáciu identifikátorov. Verzovanie kľúčov umožňuje rotáciu s prechodným obdobím dual-acceptance[3].

2.5 Zásady návrhu bezpečných protokolov

Pri praktickom bezpečnostnom protokole o výsledku často rozhodne nie výber algoritmu, ale obyčajná implementačná voľba: nejednoznačný formát, nevhodná chybová hláška alebo príliš dôverčivý parser[30].

2.5.1 Fail-closed správanie

Ak zlyhá autentifikácia, validácia vstupu, anti-replay kontrola alebo interná konzistencia dát, výsledkom musí byť zamietnutie požiadavky[16]. Systém neusiluje o „záchranu“ nejednoznačnej situácie — tento prístup je menej pohodlný pri ladení, no bezpečnejší v prevádzke. V prístupových systémoch je fail-closed obzvlášť dôležitý, pretože nesprávne povolenie prístupu má vyššiu cenu než nesprávne zamietnutie.

2.5.2 Jednoznačná serializácia a validácia vstupu

Pri podpisovaní je kritické, aby všetky strany pracovali s rovnakou reprezentáciou dát[26]. Ak by odosielateľ podpisoval dáta v inom formáte než príjemca, mohlo by dôjsť k chybnému odmietnutiu alebo k obídenu podpisu. Vstupná správa sa odporúča spracovať v dvoch krokoch: syntaktická validácia (správny formát, maximálna veľkosť) a semantická validácia (povinné polia, rozsahy, typy)[16]. Až po úspešnej validácii sa vykonávajú náročnejšie operácie (dešifrovanie, databázové dotazy).

2.5.3 Mapovanie bezpečnostných cieľov na mechanizmy

V bezpečnom prístupovom systéme sa jednotlivé bezpečnostné ciele realizujú konkrétnymi kryptografickými a protokolovými mechanizmami[25]:

Autentickosť požiadavky je zabezpečená pomocou HMAC s tajným kľúčom[24]. Každá požiadavka od zariadenia obsahuje kryptografický podpis, ktorý server overí pred akýmkoľvek ďalším spracovaním. Bez platného podpisu je požiadavka okamžite zamietnutá (fail-closed), čím sa bráni spoofingu.

Integrita metadát (identifikátor zariadenia, cieľové dvere, sekvenčné číslo) je chránená mechanizmom AEAD AAD[23]. Metadáta vstupujú do AEAD operácie ako Associated Authenticated Data — zmena ktoréhokoľvek poľa spôsobí zlyhanie overenia autentifikačného tagu, aj keď šifrovaný obsah zostane neporušený.

Dôvernosť zabezpečuje AEAD šifrovanie[26]. Obsah správy (identifikátor karty, akcia) je šifrovaný a bez správneho kľúča ho nie je možné prečítať.

Ochrana proti replay funguje na dvoch úrovniach: monotónne sekvenčné číslo na strane zariadenia a sliding window na strane servera[16]. Zachytená platná požiadavka nemôže byť úspešne zopakovaná.

Súkromie identifikátorov v databáze je zabezpečené pseudonymizáciou — namiesto surových identifikátorov sa ukladá HMAC hash s pepperom odvodeným z master kľúča[15]. Únik databázy tak priamo neodhalí identifikátory kariet.

Detekcia manipulácie auditnej stopy používa HMAC hash chain[32]: každý záznam obsahuje HMAC predošlého záznamu. Audit anchor chráni proti truncation útoku. Verifikácia prebieha rekonštrukciou reťaze a kontrolou konzistencie voči anchoru.

2.5.4 Kompromisy medzi bezpečnosťou a použiteľnosťou

Prístupové systémy musia byť použiteľné: rýchla odozva, nízky počet falošných zamietnutí a jednoduchá správa oprávnení[30]. To vytvára kompromisy. Napríklad príliš úzka anti-replay window môže spôsobovať zamietnutia pri nestabilnej sieti;

príliš široká window zas predlžuje časové okno, počas ktorého je zachytená požiadavka potenciálne zopakovateľná. Bezpečnostné parametre (veľkosť okna, časová tolerancia) by mali byť konfigurovateľné a odvodené od prevádzkových podmienok[16].

2.6 Správa kryptografických kľúčov

Aj jednoduchý systém potrebuje vedieť, odkiaľ sa kľúče berú, na čo slúžia, kedy sa menia a čo sa stane po incidente[3].

2.6.1 Oddelenie kľúčov podľa účelu

Dobrá prax je používať odlišné kľúče pre odlišné účely: kľúč pre HW autentifikáciu (HMAC), kľúč pre AEAD šifrovanie, kľúč pre audit hash chain a pepper pre hashovanie identifikátorov. Použitie jedného kľúča pre viac účelov môže viesť k cross-protocol útokom. Podkľúče sa odvodlia z hlavného tajomstva pomocou HKDF[19].

2.6.2 Verzovanie a rotácia kľúčov

Každé zariadenie má priradenú verziu kľúča. Pri rotácii sa nastaví nová verzia a server môže dočasne akceptovať aj predchádzajúcu verziu (*allow-previous*), aby sa minimalizoval výpadok pri prechode[3].

2.6.3 Ukladanie kľúčov a provisioning

V jednoduchších nasadeniach sa kľúče ukladajú v konfiguračných súboroch alebo v NVS pamäti zariadenia. V produkčnom prostredí sa odporúča použiť HSM alebo vault[3]. Provisioning zariadení typicky používa predzdieľané tajomstvo (PSK) alebo bootstrap protokol na báze certifikátov.

2.6.4 Kompromitácia a postup obnovy

Aj pri dobrej implementácii treba počítať s kompromitáciou zariadenia[3]. Incident plán by mal riešiť rýchle zablokovanie kompromitovaného zariadenia (karanténa), rotáciu kľúčov a identifikáciu udalostí, ktoré mohli byť ovplyvnené kompromitáciou.

2.7 Ochrana súkromia identifikátorov

Surové identifikátory kariet by sa v databáze nemali ukladať v čitateľnej forme. Štandardným riešením je pseudonymizácia: namiesto surového UID sa ukladá $\text{HMAC}(K_{\text{pepper}}, \text{UID})$, kde pepper je odvodený z master key cez HKDF[19]. Pri rotácii pepperu sa HMAC hodnoty aktualizujú. Únik databázy tak priamo neodhalí identifikátory kariet[15].

2.8 Tamper-evident audit log

Audit v prístupovom systéme nemá slúžiť iba na to, aby si administrátor pozrel, kto prišiel ku dverám. Má byť zdrojom dôkazu po incidente[32, 18]. Základné typy útokov na logy: modifikácia záznamu, vloženie falošnej udalosti, vymazanie/truncation a zmena poradia.

2.8.1 Hash chain s HMAC

Štandardným riešením je HMAC hash chain[32]: každý záznam obsahuje HMAC hash predošlého záznamu:

$$H_i = \text{HMAC}(K_{\text{audit}}, \text{encode}(\text{record}_i) \parallel H_{i-1}) \quad (2.2)$$

kde K_{audit} je tajný auditný kľúč odvodený cez HKDF. Zmena záznamu spôsobí nekonzistenciu v následných hodnotách.

2.8.2 Audit anchor a ochrana proti truncation

Hash chain sama o sebe nezabráni útoku, pri ktorom útočník odstráni posledných n záznamov a ponechá konzistentný prefix. Riešením je audit anchor: periodicky sa ukladá hodnota H_i a index i do samostatnej štruktúry. Pri verifikácii sa kontroluje, že audit log obsahuje všetky záznamy minimálne po posledný anchor.

2.8.3 Forward integrity

Pojem *forward integrity*[32] znamená, že aj pri kompromitácii kľúča v čase t by útočník nemal byť schopný spätne upraviť staršie záznamy. Jednoduchý hash chain s fixným kľúčom poskytuje detekciu modifikácie, ale pri kompromitácii kľúča možno vytvoriť alternatívnu históriu. Pre produkčný systém by bolo vhodné zvážiť evolúciu kľúča po epochách.

2.8.4 Verifikácia

Verifikácia prebieha offline alebo cez admin panel: načítanie záznamov, rekonštrukcia reťaze, kontrola konzistencie H_i a kontrola anchor proti truncation. Výstupom je index prvého miesta, kde reťaz zlyhá.

2.9 Bezpečnosť vstavaného zariadenia

Embedded zariadenie (čítačka) je z bezpečnostného hľadiska typicky najzraniteľnejším komponentom prístupového systému[31]: útočník k nemu môže mať fyzický prístup a firmvér obsahuje tajomstvá potrebné pre komunikáciu so serverom.

2.9.1 Ukladanie tajomstiev na zariadení

Tajné kľúče (napr. HMAC pre autentifikáciu) sú na embedded zariadeniach typicky uložené v NVS (Non-Volatile Storage) alebo v secure element[31]. Pri kompromitácii zariadenia sa predpokladá, že tajomstvo je odhalené[16]. Bezpečnostný návrh preto nekladie celú bezpečnosť na stranu zariadenia: server overuje každú požiadavku

nezávisle a pri anomáliách môže zariadenie zakaranténovať. Rotácia kľúčov umožňuje obmedziť dopad kompromitácie[3].

2.9.2 Monotónne počítadlá a trvácnosť stavu

Zariadenie udržiava monotónne sekvenčné počítadlo v perzistentnom úložisku, ktoré prežíva reštart. Monotónnosť je kritická: ak by sa počítadlo resetovalo, útočník by mohol znovu použiť staré platné hodnoty[16]. Platformy ako ESP32 podporujú atomické zápisy do NVS, čo znižuje riziko korupcie pri výpadku napájania[7].

2.9.3 Fyzické útoky a Wi-Fi konfigurácia

Fyzický prístup k čítačke umožňuje extrakciu firmvéru, odpočúvanie zberníc alebo pripojenie na debugging rozhranie[31]. Tieto riziká sa kompenzujú na serverovej strane: server nikdy nedôveruje zariadeniu viac, než dovoľuje overenie autentifikácie a anti-replay kontrola[30]. Wi-Fi konfigurácia (SSID, heslo, adresa servera) je uložená v perzistentnom úložisku; pre produkčné nasadenie sa odporúča WPA3 a certifikátová autentifikácia[28].

2.10 Bezpečnosť transportnej vrstvy

V jednoduchších nasadeniach je požiadavka od zariadenia autentifikovaná (integrita a pôvod cez HMAC), ale dôvernosť prenosu identifikátorov na úrovni HTTP nemusí byť kryptograficky chránená. TLS túto medzeru uzatvára na transportnej úrovni.

2.10.1 Prečo nestačí iba TLS

TLS (Transport Layer Security) chráni komunikáciu medzi dvoma koncovými bodmi, no samo o sebe neposkytuje ochranu proti replay na aplikačnej úrovni, autentifikáciu zariadenia voči serveru (bez klientských certifikátov), ani integritu interných metadát viazanú na obsah správy. Preto sú HMAC podpis, anti-replay sekvencie a AEAD frame komplementárne k TLS, nie jeho náhradou[28, 35].

2.10.2 Praktické nasadenie TLS v IoT

Na embedded platformách je TLS realizovateľné cez knižnice ako mbedTLS[28]. Hlavné výzvy: správa certifikátov na zariadeniach s obmedzenou pamäťou, certificate pinning (ochrana pred MITM aj pri kompromitovanom CA) a mTLS (vzájomná autentifikácia server–zariadenie)[35]. V systémoch s predzdieľanými symetrickými tajomstvami je HMAC autentifikácia pragmatickým ekvivalentom mTLS bez nutnosti PKI infraštruktúry[24].

2.11 Súvisiace prístupy a postavenie navrhovaného riešenia

Prístupové systémy v praxi existujú v rôznych topológiách: centralizované (server rozhoduje, čítačka len zbiera) vs. distribuované (kontrolér pri dverách rozhoduje lokálne), online vs. offline režim, cloud vs. on-premise nasadenie.

2.11.1 Komunikačné protokoly: Wiegand a OSDP

Historicky najrozšírenejším rozhraním medzi čítačkou a kontrolérom je protokol Wiegand. Ide o jednosmerný, nešifrovaný prenos identifikátora po dvoch vodičoch (Data0, Data1). Wiegand neposkytuje autentifikáciu, integritu ani ochranu proti replay — útočník s fyzickým prístupom k vedeniu môže zachytiť UID a neskôr ho zopakovať. Napriek tomu zostáva v mnohých inštaláciách nasadený kvôli jednoduchosti a spätnej kompatibilite.

Modernejšou alternatívou je OSDP v2 (Open Supervised Device Protocol)[17], ktorý pridáva Secure Channel s AES-128 šifrovaním, MAC autentifikáciou a sekvenčnými číslami. OSDP v2 rieši väčšinu nedostatkov Wiegand, no jeho nasadenie vyžaduje podporu na oboch stranách (čítačka aj kontrolér) a správu symetrických kľúčov.

2.11.2 Postavenie navrhovaného riešenia

Navrhované riešenie spadá do kategórie *centralizovaný, online, on-premise*[8]: server vyhodnocuje každú požiadavku v reálnom čase, čo umožňuje plnohodnotnú anti-replay ochranu a runtime anomálnu detekciu. Výhodou centralizovaného prístupu je jednoduchšia správa politík a kľúčov; nevýhodou je závislosť na dostupnosti servera.

V porovnaní s Wiegand[9] navrhovaný prístup poskytuje kryptografickú autentifikáciu (HMAC), integritu (AEAD + AAD) a anti-replay ochranu. V porovnaní s OSDP v2[33] pridáva HKDF per-reader deriváciu kľúčov (izolácia medzi čítačkami), deterministický nonce s runtime verifikáciou a tamper-evident audit s HMAC hash chain — mechanizmy, ktoré v štandardnom OSDP nie sú prítomné[25, 30].

2.12 Zhrnutie teoretických východísk

Bezpečnosť prístupového systému stojí na previazaných vrstvách: autentifikácia požiadaviek je zbytočná bez anti-replay ochrany; anti-replay bez autentifikácie umožňuje útočníkovi generovať nové požiadavky; AEAD bez AAD nezabezpečuje integritu metadát; audit bez kryptografického reťazenia nemá dôkazovú hodnotu po incidente. Žiadny jednotlivý mechanizmus neposkytuje úplnú ochranu — systém je bezpečný len ako celok[25, 16].

2.12.1 Predpoklady a zvyškové riziko

Každý bezpečnostný návrh implicitne stojí na predpokladoch[16]. Typicky sa predpokladá, že server je lepšie chránený než čítačka a že administrátorské rozhranie je prístupné iba autorizovaným osobám. Aj pri splnení predpokladov ostáva zvyškové riziko (residual risk): kompromitácia jednej čítačky, dlhodobý DoS alebo insider útok. Dôležité je, aby systém v takom prípade zlyhal bezpečne (fail-closed) a poskytol auditné stopy pre analýzu[32].

Vrstvené zabezpečenie (defense-in-depth)[25] znamená, že systém nestojí

na jednom mechanizme. Autentifikácia chráni pôvod požiadavky, anti-replay chráni pred opakovaním, AEAD chráni správy a integritu metadát, tamper-evident audit chráni dôveryhodnosť histórie a anomálna detekcia eskaluje reakciu pri opakovaných incidentoch. Konkrétne prepojenie medzi týmito teoretickými konceptmi a experimentálnym overením je súčasťou praktickej časti práce.

3 Realizácia a experimentálne vyhodnotenie

Táto kapitola popisuje praktickú realizáciu navrhnutého systému a experimentálne vyhodnotenie jeho bezpečnostných vlastností. Prvá podkapitola sa venuje samotnej implementácii prototypu — štruktúre projektu, toku spracovania udalosti a hlavným bezpečnostným mechanizmom v kóde. Druhá podkapitola systematicky hodnotí šesť konfiguračných profilov v siedmich útočných scenároch a porovnáva ich s referenčným nasadením podľa RFC 8439.

3.1 Praktická implementácia navrhnutého systému

Táto kapitola vychádza z konkrétnej implementácie projektu `access-security`. Kódová báza je rozdelená do samostatných modulov, ktoré riešia kryptografiu, formát správ, rozhodovaciu logiku, úložisko, audit a administrátorské rozhranie. Pre správnu interpretáciu je potrebné najprv presne popísať, čo je v prototypu skutočne implementované, kde prebieha rozhodovanie a ktoré časti systému nesú bezpečnostnú zodpovednosť.

Opis implementácie sa člení na dve vrstvy. Prvá vrstva opisuje samotnú implementáciu, teda štruktúru projektu, tok spracovania udalosti a hlavné mechanizmy použité v kóde. Druhá vrstva má analytický charakter: venuje sa testom, meraniam a vyhodnoteniu toho, či implementované opatrenia naozaj zvyšujú odolnosť systému voči zvoleným hrozbám.

Každé implementačné rozhodnutie vychádza z bezpečnostného princípu diskutovaného v teoretickej časti. Nasledujúci prehľad sumarizuje, kde v kóde sa realizuje každý z teoreticky opísaných mechanizmov.

Autentifikácia hardvérových požiadaviek je realizovaná v module `access_admin` (endpoint `/api/hw/uid`): každá HTTP požiadavka od čítačky obsahuje hlavičku `X-HW-Signature` s HMAC-SHA-256 podpisom nad reťazcom `"POST /api/hw/uid\n"` + telo požiadavky (zahrnutie metódy a cesty oddelených znakom LF chráni proti cross-endpoint replay). Server podpis overí pred akýmkoľvek

ďalším spracovaním. Kľúč pre HMAC je oddelený od ostatných kryptografických účelov.

AEAD šifrovanie a integrita interného frame zabezpečujú moduly `crypto_lib` a `access_core`. Po overení HMAC server nevyhodnocuje požiadavku priamo, ale vytvára z nej interný bezpečný frame: binárnu štruktúru, kde je payload zašifrovaný algoritmom ChaCha20-Poly1305 alebo XChaCha20-Poly1305. Hlavička frame (vrátane `reader_id`, `door_id`, `seq`, `key_version` a nonce) vstupuje do AEAD operácie ako AAD, čím je kryptograficky viazaná na šifrovaný obsah.

Derivácia kľúčov prebieha v module `key_manager` pomocou HKDF-SHA-256 s domain separation: z jedného master secret sa odvodí oddelené podkľúče pre AEAD, nonce generovanie, auditný hash chain a pepper pre pseudonymizáciu identifikátorov kariet. Každá čítačka má vlastný AEAD kľúč (salt obsahuje `reader_id`), čo zabezpečuje kryptografickú izoláciu medzi zariadeniami.

Anti-replay ochrana funguje na dvoch úrovniach: na strane zariadenia firmvér udržiava monotónne `hw_seq` (modul `protocol_lib`), na strane servera sliding window sleduje použité sekvenčné čísla per čítačku. Deterministický nonce je implementovaný v module `crypto_lib`.

Anomálna detekcia a karanténa sú realizované komponentom anomálneho detektora v module `access_core`. Detektor sleduje per-reader stav a pri podozrivej aktivite (opakovaný seq, rollback, nonce mismatch, séria zlyhaní tagu) zariadenie proaktívne zakaranténuje.

Pseudonymizácia identifikátorov kariet v module `access_storage` zabezpečuje, že databáza neobsahuje surové UID; ukladá sa HMAC hash s pepperom odvodeným z master kľúča.

Tamper-evident audit log (taktiež `access_storage`) používa HMAC hash chain s anchor ochranou proti truncation.

RBAC autorizácia v module `access_decision` overuje väzbu medzi čítačkou, dverami a rolou karty.

3.1.1 Architektúra implementácie

Implementácia je koncipovaná ako modulárny serverovo-riadený prototyp s oddelenou rozhodovacou logikou. Na strane zariadenia sa nachádza čítačka postavená na platforme ESP32[7] s modulom RC522, komunikujúca podľa ISO 14443[14]. Jej úloha je úmyselne obmedzená: načíta UID karty, pripojí identifikátor čítačky a dverí, doplní monotónne počítadlo `hw_seq` a takto vytvorenú požiadavku autentifikuje pomocou HMAC-SHA256. Samotné rozhodovanie o prístupe neprebíha na zariadení, ale na serveri.

Serverová časť je implementovaná v jazyku C++20 a pozostáva z dvoch nezávislých HTTP služieb:

- **Frontend (port 8080):** modul `access_admin` — prijíma požiadavky od hardvéru (`/api/hw/uid`), overuje HMAC podpis, vytvára interný AEAD frame a odošle ho na rozhodovací komponent. Zároveň poskytuje administratívne API a webové rozhranie.
- **DecisionService (port 8081):** endpoint `/api/decision/frame` — prijíma raw AEAD frame bytes a spúšťa pipeline v poradí od najlacnejších kontrol: parsovanie → validácia čítačky → verzia kľúča → anti-replay → AEAD dešifrovanie → anomálna detekcia → RBAC → audit. Beží ako samostatná služba s oddeleným mutex-om (`engineMutex`).

Toto rozdelenie zabezpečuje, že AEAD frame vždy prechádza cez HTTP rozhranie — rovnaký kanál, aký používajú E2E experimenty. Architektúra je navrhnutá ako distribuovaná: v predvolenom režime (`decision_mode: remote`) frontend a DecisionService komunikujú cez HTTP a môžu bežať na rovnakom alebo na oddelených serveroch bez zmeny rozhrania. DecisionService je viazaný výhradne na loopback rozhranie (`127.0.0.1`), čím je sieťovo izolovaný — prístup k nemu má len lokálna frontend služba[16]. SQLite používa WAL mód s busy timeout pre bezpečný súbežný prístup. Pri nasadení na oddelených serveroch by každá služba mala vlastné úložisko. Tabuľka 3–1 sumarizuje hlavné vrstvy a ich úlohu.

Tabuľka 3 – 1 Hlavné vrstvy implementácie a ich úloha v prototype

Vrstva	Úloha
Hardvérová vrstva	ESP32 + RC522 načíta UID karty, vytvorí JSON požiadavku, zvýši lokálne počítadlo <code>hw_seq</code> a vypočíta HMAC podpis.
Frontend (port 8080)	Modul <code>access_admin</code> : endpoint <code>/api/hw/uid</code> (overenie HMAC, konštrukcia AEAD frame), administratívne API, webové rozhranie. Frame sa odošle na <code>DecisionService</code> cez HTTP.
<code>DecisionService</code> (port 8081)	Endpoint <code>/api/decision/frame</code> : prijíma raw frame bytes, spúšťa <code>DecisionEngine</code> (AEAD decrypt, anti-replay, anomálna detekcia, RBAC). Beží ako samostatná služba.
Úložisko a audit	Modul <code>access_storage</code> : SQLite (WAL mód) pre konfiguráciu a HMAC-chained auditný log s anchor ochranou.
Pozorovanie	Modul <code>runtime_events</code> : prevádzkové udalosti zobrazované v administrátorském rozhraní.

3.1.2 Modulárna štruktúra a externé závislosti

Projekt je rozdelený do deviatich modulov s jednoznačne definovanými závislosťami: `crypto_lib` (AEAD, HKDF, generátory nonce, HMAC utility), `protocol_lib` (formát frame, replay window), `key_manager` (HKDF derivácia, rotácia), `access_core` (frame handler, anomálny detektor), `access_decision` (rozhodovacia logika), `access_storage` (SQLite, audit log), `access_admin` (HTTP server, služby), `config_loader` (YAML konfigurácia) a `runtime_events` (prevádzkové udalosti).

Kryptografické operácie na serveri sú realizované výhradne cez knižnicu libsodium[6] — implementáciu štandardizovaných algoritmov (ChaCha20-Poly1305 podľa RFC 8439[26], HMAC-SHA-256 podľa FIPS 198-1[24], HKDF podľa RFC 5869[19]), ktorá je nezávisle auditovaná[27], implementuje všetky operácie v konštantnom čase a je kompilovaná zo zdrojového kódu ako súčasť CMake build procesu. Na strane ESP32 je HMAC autentifikácia realizovaná cez mbedTLS (súčasť ESP-IDF).

3.1.3 AEAD šifrovanie interného frame

Po úspešnom overení hardvérovej požiadavky server nevyhodnocuje prístup priamo z prijatých dát. Namiesto toho z nich vytvára interný bezpečný frame: binárnu štruktúru, v ktorej je payload zašifrovaný algoritmom AEAD a hlavička je kryptograficky previazaná cez mechanizmus Associated Authenticated Data. Parser frame pri deserializácii kontroluje maximálnu dĺžku ciphertextu (`maxCtLen` = 4096 B) ešte pred alokáciou pamäte, čím chráni proti memory-exhaustion útoku cez nadmerne veľké frame.

Voľba algoritmu Implementácia podporuje dva AEAD algoritmy z rodiny ChaCha20-Poly1305, medzi ktorými je možné prepínať prostredníctvom konfigurácie (`cipher_mode`). Oba sú postavené na rovnakom kryptografickom jadre (ChaCha20 stream cipher + Poly1305 MAC) a oba so 256-bitovou kľúčovou bezpečnosťou:

- **XChaCha20-Poly1305** s 192-bitovým noncom (24 bajtov). Rozšírený nonce je konštruovaný pomocou medzikroku HChaCha20, vďaka čomu je bezpečné používať náhodne generované nonce aj pod dlhodobu žijúcim kľúčom: pravdepodobnosť kolízie pri 2^{96} správach je zanedbateľná[1, 21].
- **ChaCha20-Poly1305 IETF** s 96-bitovým noncom (12 bajtov). Štandardizovaný v RFC 8439 a široko nasadzovaný v TLS 1.3 aj v ďalších protokoloch[26]. Kratší nonce znižuje bezpečný limit pre náhodne generované nonce na približne 2^{48} správ.

Implementácia podporuje tri stratégie generovania nonce (náhodný, deterministický HMAC a deterministický s runtime verifikáciou). Pri deterministickej konštrukcii je riziko kolízie pre oba algoritmy identické: nonce sa zopakuje iba vtedy, keď sa zopakuje vstupný kontext, čomu bráni kombinácia anti-replay mechanizmu a unikátneho `seq`. Oba algoritmy sú preto zahrnuté v implementácii nie kvôli rozdielnej bezpečnosti, ale kvôli možnosti porovnať ich výkon a režijné náklady.

Konštrukcia nonce Implementácia definuje rozhranie `INonceGenerator` s jednou metódou `generate(context, seq)`, za ktorým stoja tri konkrétne stratégie:

Náhodný nonce. Trieda `RandomNonceGenerator` ignoruje kontext aj sekvenčné číslo a pri každom volaní vygeneruje kryptograficky bezpečný náhodný nonce pomocou `randombytes_buf`:

$$\text{nonce}_{\text{rand}} = \text{randombytes_buf}(n) \quad n \in \{12, 24\} \quad (3.1)$$

Bezpečnosť tejto stratégie závisí na kvalite generátora náhodných čísiel a na počte správ pod jedným kľúčom. Pre XChaCha20 s 24-bajtovým noncom je riziko kolízie zanedbateľné do 2^{96} správ; pre ChaCha20 s 12-bajtovým noncom klesá bezpečná hranica na približne 2^{48} .

Deterministický HMAC nonce. Trieda `HmacNonceGenerator` odvodzuje nonce deterministicky z dedikovaného kľúča K_{nonce} a kontextu hlavičky frame. Kľúč sa odvodzuje cez HKDF-SHA-256 z master kľúča:

$$K_{\text{nonce}} = \text{HKDF-SHA-256}(K_{\text{master}}, \text{reader_id} \parallel \text{key_version}, \text{"nonce-derivation-v1"}) \quad (3.2)$$

Samotný nonce sa počíta ako skrátený výstup HMAC-SHA-256 nad serializovanou hlavičkou (52 bajtov), v ktorej je pole nonce vynulované, aby sa predišlo kruhovej závislosti:

$$\text{nonce}_{\text{det}} = \text{HMAC-SHA-256}(K_{\text{nonce}}, \text{header_context})[:n] \quad n \in \{12, 24\} \quad (3.3)$$

kde $\text{header_context} = \text{reader_id} \parallel \text{door_id} \parallel \text{ts_unix_ms} \parallel \text{seq} \parallel \text{key_version} \parallel 0^{24}$ v little-endian formáte.

Deterministická konštrukcia znižuje závislosť na kvalite generátora náhodných čísiel[4]. Nonce sa zopakuje iba vtedy, keď sa zopakuje celý kontext, čomu bráni unikátnosť `seq` garantovaná anti-replay mechanizmom. Navyše, keďže kontext obsahuje aj `door_id` a `ts_unix_ms`, rovnaký nonce nevznikne ani pri rôznych kontextoch s rovnakým sekvenčným číslom.

Rozdiel medzi deterministickým nonce bez a s verifikáciou. Obe deterministické stratégie používajú identický `HmacNonceGenerator`. Odlišuje ich to, či server po vytvorení frame *overuje*, že prijatý nonce zodpovedá deterministickému výpočtu. Overenie vykonáva `ProtocolAnomalyDetector` (sekcia 3.1.6), ktorý je aktívny iba pri zapnutej anomálnej detekcii. Bez aktívneho detektora sa nonce síce generuje deterministicky, ale server akceptuje frame bez verifikácie nonce zhody.

Na drôtovom formáte frame zaberá pole nonce vždy 24 bajtov kvôli uniformnej štruktúre hlavičky. Pri `XChaCha20-Poly1305` sa využíva celých 24 bajtov; pri `ChaCha20-Poly1305` sa používa prvých 12 bajtov a zvyšných 12 je nulových. Táto konvencia zjednodušuje parsovanie a zaručuje, že veľkosť hlavičky je rovnaká naprieč všetkými konfiguráciami.

AAD väzba hlavičky Hlavička frame obsahuje polia `reader_id`, `door_id`, `ts_unix_ms`, `seq`, `key_version` a `nonce`. Tieto metadáta sa nešifrujú, ale vstupujú do AEAD operácie ako AAD. Výsledkom je, že zmena ktoréhokoľvek z nich

spôsobí zlyhanie overenia autentifikačného tagu, aj keď šifrovaný payload zostane neporušený[23].

Praktický význam tejto väzby sa ukáže pri konkrétnom útoku. Bez AAD by útočník mohol zachytiť platný frame a podvrhnúť hodnotu `door_id`: payload by bol stále korektne dešifrovaný, no systém by vyhodnotil prístup pre iné dvere. AAD toto znemožňuje, pretože kryptograficky viaže šifrovaný obsah na presný kontext metadát. Ide o realizáciu princípu, že kontext autorizácie musí byť nedeliteľnou súčasťou správy.

Implementácia podporuje aj konfigurovateľný režim `aad_mode: "none"`, v ktorom sa AAD nepoužíva a hlavička nie je kryptograficky chránená. Tento režim je v kóde prítomný ako záložná možnosť; všetky testované konfigurácie používajú plnú AAD väzbu (`aad_mode: "full"`).

3.1.4 Derivácia kľúčov a domain separation

Celý kryptografický materiál v prototype pochádza z jedného 256-bitového master secret, uloženého v súbore `secrets/master_key.hex` mimo repozitára. Manuálne spravovaný tajný súbor je pragmatický kompromis vhodný pre prototyp; v produkčnom nasadení by jeho úlohu plnil HSM alebo vault[3]. Z tohto tajomstva sa pomocou HKDF-SHA-256[19] deterministicky odvodí tri nezávislé skupiny podkľúčov:

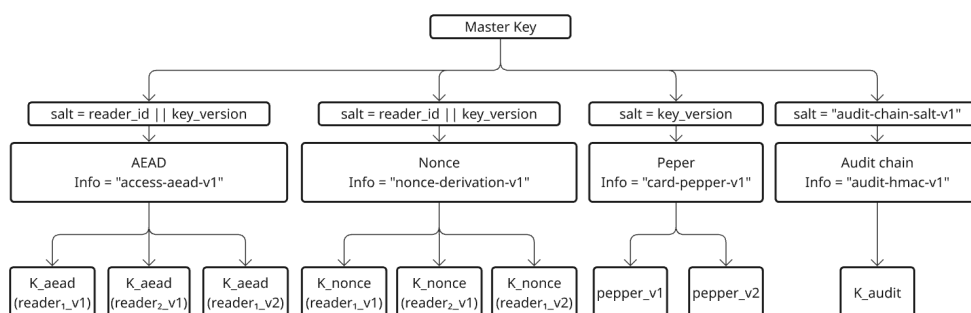
1. **AEAD kľúče** pre šifrovanie interného frame. Salt je zložený z `reader_id` (4 bajty, little-endian) a `key_version` (4 bajty), info reťazec je `"access-aead-v1"`.
2. **Nonce kľúče** pre deterministickú deriváciu nonce. Salt je rovnaký ako pri AEAD kľúčoch (`reader_id || key_version`), info reťazec je `"nonce-derivation-v1"`.
3. **Pepper** pre hashovanie identifikátorov kariet. Salt obsahuje iba `key_version`, info reťazec je `"card-pepper-v1"`.
4. **Auditný HMAC kľúč** pre hash chain. Salt je `"audit-chain-salt-v1"`, info reťazec `"audit-hmac-v1"`.

Tabuľka 3–2 sumarizuje derivačnú schému.

Tabuľka 3–2 Prehľad HKDF derivácie kľúčov z master secret

Kľúč	Salt	Info reťazec
AEAD	reader_id key_ver	access-aead-v1
Nonce (K_{nonce})	reader_id key_ver	nonce-derivation-v1
Pepper	key_ver	card-pepper-v1
Auditný HMAC	audit-chain-salt-v1	audit-hmac-v1

Grafické znázornenie tejto derivačnej hierarchie je v obrázku 3–1.



Obrázok 3–1 Derivácia kľúčov z master secret cez HKDF. Každá vetva má vlastný info reťazec (domain separation).

Info reťazce zabezpečujú domain separation[19]: z rovnakého master kľúča nikdy nevznikne rovnaký podkľúč pre dva rôzne účely, ani keby sa hodnoty `reader_id` a `key_version` náhodne prekrývali. Súčasne umožňujú neskoršiu zmenu verzie schémy (napríklad z `access-aead-v1` na `access-aead-v2`) bez spätnej kompatibility so starými kľúčmi, čo je dôležité pri migračných scenároch.

Z bezpečnostného hľadiska je podstatné, že AEAD kľúč sa odvodzuje per-reader. Kompromitácia jedného zariadenia a extrakcia jeho odvoditeľného kľúča neodhaľuje kľúče iných čítačiek, pretože salt HKDF obsahuje unikátnu hodnotu

`reader_id`. Izolácia je priamym dôsledkom správnej konštrukcie derivácie a nevyžaduje distribúciu samostatných tajomstiev.

Verzovanie kľúčov v salt zabezpečuje, že pri rotácii (zmene `key_version`) sa odvodí úplne nový AEAD kľúč bez nutnosti meniť master secret. Server v konfigurácii umožňuje režim `allow_previous_key_version`, v ktorom dočasne akceptuje aj bezprostredne predchádzajúcu verziu. Cieľom je hladký prechod pri rotácii: zariadenie nemusí byť aktualizované presne v rovnakom okamihu ako server, čo znižuje riziko krátkodobého výpadku[3].

Implementácia podporuje aj priamy režim derivácie (bez HKDF); všetky testované konfigurácie používajú HKDF s per-reader salt, čím je zaručená izolácia kľúčov medzi čítačkami.

3.1.5 Anti-replay mechanizmus

Anti-replay ochrana funguje na dvoch úrovniach. Na strane zariadenia firmvér udržiava monotónne počítadlo `hw_seq`, uložené v NVS pamäti ESP32, ktoré sa inkrementuje pri každom skene karty a prežíva reštart zariadenia. Každá požiadavka tak nesie unikátnu sekvenčnú hodnotu.

Na strane servera modul `protocol_lib` implementuje štruktúru `ReplayWindow`, ktorá udržiava záznam o naposledy videných sekvenciách pre každú čítačku zvlášť. Mechanizmus rozlišuje tri stavy:

- ak je `seq` menšie alebo rovné `maxSeen - windowSize`, požiadavka sa odmietne ako príliš stará,
- ak `seq` spadá do okna, ale už bolo zaznamenané, odmietne sa ako *replay*,
- inak sa `seq` akceptuje, zaznamená do množiny videných hodnôt a posunie sa horná hranica okna.

Prečo nestačí jednoduchá podmienka `seq > lastSeen`? V prostredí Wi-Fi nie je zaručené, že dve po sebe nasledujúce požiadavky dorazia na server v rovnakom

poradí. Bez okna by druhá požiadavka bola nesprávne odmietnutá, aj keď by bola legitímna. Sliding window toleruje krátke zmeny poradia, pokiaľ správa ešte nebola videná, a tým znižuje podiel falošných odmietnutí pri zachovaní replay ochrany. Dôležitou vlastnosťou je, že `ReplayWindow::remember()` sa volá až po úspešnom AEAD dešifrovaní — počiatočná kontrola `contains()` je len čítacia. Útočník tak nemôže otráviť replay window nevalidnými frame (so správnym `seq`, ale poškodeným ciphertextom).

Veľkosť okna (`replay_window_size`) je konfigurovateľná a priamo ovplyvňuje kompromis medzi bezpečnosťou a praktickou použiteľnosťou: príliš malé okno zvyšuje falošné odmietnutia, príliš veľké predlžuje čas, počas ktorého je zachytená stará požiadavka potenciálne zopakovateľná. Implementácia udržiava okno pomocou fronty (`std::deque`) a množiny (`std::unordered_set`), pričom pri prekročení kapacity sa najstaršia hodnota automaticky evikvuje.

3.1.6 Detektor protokolových anomálií

Nad rámec reaktívneho odmietnutia jednotlivých frame implementácia obsahuje komponent `ProtocolAnomalyDetector`, ktorý sleduje vzory správania na úrovni celej čítačky a pri podozrivej aktivite dokáže čítačku proaktívne umiestniť do karantény. Tento prístup vychádza z princípov detekcie anomálií v sieťovom prostredí[31].

Detektor udržiava per-reader stavový záznam a sleduje štyri typy anomálií uvedené v tabuľke 3–3.

Tabuľka 3–3 Typy anomálií sledovaných detektorom `ProtocolAnomalyDetector`

Kód anomálie	Popis
<code>seq_reuse</code>	Opakované sekvenčné číslo: frame nesie <code>seq</code> , ktoré už bolo zaznamenané — detektor udalosť registruje ako indikátor replay útoku.
<code>seq_rollback</code>	Rollback sekvenčného čísla: <code>seq + rollbackThreshold < maxSeenSeq</code> ; skok späť signalizuje pokus obísť sliding window.
<code>nonce_mismatch</code>	Nesúlad nonce: prijatý nonce nezodpovedá deterministicky odvodennej hodnote z kontextu hlavičky (aktívne iba v deterministickom móde).
<code>tag_fail_streak</code>	Séria zlyhaní tagu: počet po sebe nasledujúcich zlyhaní AEAD tagu dosiahne konfigurovateľný limit, čo indikuje systematický pokus o predloženie neplatného ciphertextu.

Pri detekcii anomálie sa čítačka umiestni do karantény: všetky ďalšie frame od tejto čítačky sú zamietnuté s dôvodom `quarantined`, bez ohľadu na ich obsah. Karanténa je proaktívna obrana — izoluje celé zariadenie, nie iba jednotlivý frame. Tým sa odlišuje od anti-replay okna (ktoré zamieta iba konkrétnu sekvenciu) a od zlyhania AEAD dešifrovania (ktoré zamieta iba frame s neplatným tagom).

Detektor je konfigurovateľný — môže byť zapnutý alebo vypnutý nezávisle od stratégie generovania nonce.

Tabuľka 3–4 sumarizuje typy anomálií a reakcie detektora.

Tabuľka 3 – 4 Konfigurácia ProtocolAnomalyDetector a reakcia na anomálie

Typ anomálie	Podmienka detekcie	Reakcia
seq_reuse	$\text{seq} \in \text{_seen}$	Okamžitá karanténa
seq_rollback	$\text{seq} + \delta < \text{maxSeenSeq}$	Okamžitá karanténa
nonce_mismatch	$\text{nonce} \neq \text{HMAC}(K_n, \text{ctx})[:n]$	Okamžitá karanténa
tag_fail_streak	5 po sebe nasledujúcich zlyhaní	Karanténa po 5. zlyhaní

3.1.7 Ďalšie bezpečnostné mechanizmy

Ochrana identifikátorov kariet. Surové UID kariet sa v databáze neukladajú; ukladá sa $\text{HMAC}(\text{pepper}, \text{UID})$, kde pepper je odvodený z master key cez HKDF. Pri rotácii pepperu sa HMAC hodnoty aktualizujú.

Tamper-evident audit. Každý auditný záznam obsahuje HMAC predošlého záznamu (hash chain). Audit anchor ukladá hodnotu H_i pri každom zápise do samostatnej štruktúry, čím chráni proti truncation. Verifikácia prebieha rekonštrukciou reťaze a kontrolou konzistencie voči anchoru[32].

RBAC a väzba čítačka–dvere. Rozhodovacia logika (DecisionEngine) overuje väzbu medzi čítačkou a dverami a rolu karty. Prístup je povolený iba ak AEAD frame je platný, anti-replay kontrola prebehla úspešne a autorizačná politika vyhovuje.

Konfigurácia. Systém podporuje dva AEAD algoritmy (ChaCha20-Poly1305 a XChaCha20-Poly1305) a tri stratégie generovania nonce (náhodný, deterministický HMAC, deterministický s anomálnym detektorom). Všetky konfigurácie zdieľajú rovnakú HKDF deriváciu, AAD väzbu, anti-replay window a audit chain.

3.2 Verifikácia kvality kódu pred experimentmi

Pred experimentálnym vyhodnotením bola vykonaná verifikácia kódovej bázy:

- **Statická analýza (clang-tidy[22]):** 16 kategórií kontrol (wildcards `bugprone-*`, `cert-*`, `clang-analyzer-*`, `performance-*` pokrývajúce viac ako 250 jednotlivých pravidiel, doplnené o konkrétne pravidlá z `cppcoreguidelines`, `misc`, `modernize` a `readability`). Výsledok: 0 nálezov po opravách.
- **Dynamická analýza (sanitizers[34]):** ASan (pamäťové chyby), UBSan (nedefinované správanie), LSan (úniky pamäte). Výsledok: 0 nálezov za behu testov.
- **Testovacia sada (Google Test[11]):** 44 testovacích prípadov pokrývajúcich kryptografiu, replay window, anomálny detektor (18 testov), auditný log a rozhodovaciu logiku. Všetky testy úspešné.

Tieto výsledky posilňujú dôveru, že experimentálne merania odrážajú správanie navrhnutých bezpečnostných mechanizmov, nie implementačné chyby.

3.3 Experimentálna analýza bezpečnostných profilov

Táto kapitola nadväzuje na teoretické východiská z oblastí AEAD, správy nonce, anti-replay ochrany, derivácie kľúčov a detekcie protokolových anomálií. Kým teoretická časť formulovala bezpečnostné požiadavky a model hrozieb, táto kapitola tieto požiadavky premieňa na merateľné tvrdenia a overuje ich experimentálne pomocou profilov $A1/A2 \times R0/R1/R2$. Jadrom analýzy nie je všeobecná bezpečnosť celého informačného systému, ale konkrétne vyhodnotenie toho, ako voľba AEAD algoritmu, nonce stratégie a runtime detektora ovplyvňuje odolnosť voči zvoleným triedam útokov. Každý scenár priamo zodpovedá jednému teoretickému konceptu: S1 overuje anti-replay, S2 AAD binding, S3a nonce enforcement, S3b kryptografickú izoláciu kľúčov (HKDF), S4 sekvenčnú čerstvosť, S5 AEAD tag verifikáciu a S7

odolnosť voči modifikácii nonce v transite (MITM na kanáli medzi frontend a DecisionService).

Predchádzajúce kapitoly opísali návrh a implementáciu bezpečnostných mechanizmov prototypu. Cieľom tejto kapitoly je experimentálne overiť, či a ako tieto mechanizmy skutočne zvyšujú odolnosť systému voči vybraným útokovým scenárom. Experimenty sú navrhnuté tak, aby pokrývali rôzne triedy hrozieb a umožnili porovnanie šiestich konfiguračných profilov.

Výber scenárov vychádza z modelu hrozieb STRIDE[12] presentovaného v teoretickej časti. Nie všetky kategórie STRIDE sú pre tento prototyp rovnako relevantné: Denial of Service je čiastočne riešený na úrovni validácie vstupov a nie je predmetom kryptografickej analýzy; Elevation of Privilege sa viaže na autorizačnú politiku, nie na nonce/AEAD mechanizmus. Tabuľka 3–5 zobrazuje, ktoré kategórie STRIDE pokrývajú jednotlivé scenáre.

Tabuľka 3–5 Mapovanie STRIDE kategórií hrozieb na experimentálne scenáre

STRIDE	Scenár	Mechanizmus	Testovaná vlastnosť
Tampering	S2	AAD binding	Zmena hlavičky frame.
Tampering	S5	AEAD Poly1305	Poškodený ciphertext.
Tampering	S7	AEAD + R2	Nonce MITM v transite.
Replay	S1	ReplayWindow	Opakovanie frame.
Nonce enforcement	S3a	R2 detektor	Odchýlka nonce.
Key isolation	S3b	HKDF + AEAD	Cross-reader frame.
Seq. rollback	S4	R2 detektor	Rollback seq čísla.
Throughput	S6	—	Výkonnostný overhead.

Scenáre S3, S4 a S6 nemajú priamu analógiu v STRIDE, ale pokrývajú hrozby špecifické pre protokoly s per-reader deriváciou kľúčov, rollback sekvencie a výkonnostný dopad mechanizmov. S3 vychádza z princípu kryptografickej

izolácie prostredníctvom HKDF[19]; S4 z literatúry o nonce misuse a odolnosti protokolov[29, 4].

3.3.1 Matica konfiguračných profilov

Experimentálny harness kombinuje dva šifrovacie algoritmy s tromi režimami generovania nonce, čo vytvára maticu šiestich profilov (tabuľka 3–6):

Tabuľka 3–6 Matica konfiguračných profilov (algoritmus $\times$ režim nonce)

	R0 (random)	R1 (determ.)	R2 (det.+detektor)
A1 (ChaCha20)	A1–R0	A1–R1	A1–R2
A2 (XChaCha20)	A2–R0	A2–R1	A2–R2

Všetky profily zdieľajú rovnaký základ:

- derivácia kľúčov cez HKDF s per-reader salt a verziovaním,
- plný AAD mód (celý plaintext header ako Additional Authenticated Data),
- anti-replay ochrana cez *ReplayWindow* s kapacitou 256,
- verzovaný pepper pre HMAC identifikátorov kariet,
- enforced reader-door binding (väzba čítačky na dvere).

Rozdiely medzi profilmi spočívajú v troch dimenziách:

Algoritmus (A1 vs. A2): ChaCha20-Poly1305 (RFC 8439[26], 12-bajtový nonce) oproti XChaCha20-Poly1305[1] (24-bajtový nonce). Väčší nonce priestor u A2 znižuje pravdepodobnosť kolízie pri náhodnom generovaní.

Režim nonce (R0): Nonce je generovaný pseudonáhodne pre každý frame. Server nonce neoveruje — používa ho len na dešifrovanie.

Režim nonce (R1): Nonce je generovaný deterministicky ako HMAC-SHA-256 kontextu hlavičky: $\text{nonce} = \text{HMAC}(K_{\text{nonce}}, \text{header_context})$, kde K_{nonce} je derivovaný cez HKDF s info `nonce-derivation-v1`. Server má rovnaký kľúč, ale nonce neoveruje.

Režim nonce (R2): Rovnaký deterministický nonce ako R1, ale s aktívnym *ProtocolAnomalyDetector*, ktorý:

- overuje, že nonce v hlavičke zodpovedá očakávanej hodnote (`nonce_mismatch`),
- detekuje rollback sekvenčného čísla (`seq_rollback`, prah $\delta = 100$),
- sleduje sériu neúspešných AEAD overení (`tag_fail_streak`, limit 5),
- pri detekcii anomálie čítačku *zakaranténuje* — všetky ďalšie frame z danej čítačky sú zamietnuté.

3.3.2 Architektúra experimentálneho harnessu

Experimentálny harness je implementovaný v jazyku C++ a zdieľa rovnaký kryptografický kód aj rozhodovaciu logiku (*DecisionEngine*) ako produkčný server. Tým je zaručené, že experimenty merajú skutočné správanie systému, nie simuláciu oddelenú od implementácie — prístup odporúčaný pre verifikáciu bezpečnostných vlastností[16]. Harness nie je unit testom izolovanej kryptografickej funkcie ani úplným end-to-end testom vrátane siete; ide o *security-oriented component test*: overuje, ako je AEAD v systéme použité — ako sa tvorí nonce, ako je hlavička viazaná cez AAD, ako systém reaguje na replay, rollback a odchýlky od protokolových invariantov.

Hlavné komponenty

ExperimentContext: Pre každú kombináciu (profil, `attack_n`, `run`) vytvára čerstvú inštanciu obsahujúcu *KeyManager*, *SqliteAccessStore*, *DecisionEngine*, *ProtocolAnomalyDetector* a mapu *ReplayWindow*. Tým sa zaručuje, že stav z predchádzajúceho behu neovplyvní výsledok.

FrameFactory: Generuje validné binárne frame identické s tým, čo by vytvoril produkčný systém. Poskytuje aj statické metódy na manipuláciu frame (`tamperDoorId`, `tamperCiphertext`, `tamperReaderId`, `tamperNonce`, `tamperSeq`).

MetricsCollector: Zapisuje výsledky do CSV s 15 stĺpcami vrátane `seed` (skutočne použitý seed pre daný beh, t.j. `base_seed + run_idx`) a `run_idx`. Každý riadok CSV tak jednoznačne identifikuje beh a umožňuje jeho nezávislú reprodukciu.

IScenario: Rozhranie pre scenáre. Každý scenár implementuje metódy `name()`, `config()`, `setup()`, `buildTrialFrame()` a voliteľne `onBaselineFrame()`.

runScenario(): Spoločná smyčka iterujúca cez 6 profilov $\times$ N krokov $\times$ 5 behov.

Fázy experimentu Každá iterácia prebieha v troch fázach:

1. **Warmup** (200 frame, nezaznamenaný): Naplní replay window a ustáli stav detektora.
2. **Baseline** (200 frame, zaznamenaný): Validné frame. Slúži na meranie false reject rate a referenčnej latencie. Baseline sa opakuje pre každú kombináciu (profil, `attack_n`, `run`) — každá iterácia beží s čerstvým kontextom.
3. **Trial / Attack** (N frame, zaznamenaný): Scenár generuje útočné alebo meracie frame cez `buildTrialFrame()`.

Reprodukovateľnosť

- Každý beh r ($r = 0 \dots 4$) používa iný seed $42 + r$, čo zavádza medzibehová variabilitu pri zachovaní reprodukovateľnosti. Režim R0 používa `SeededNonceGenerator` s týmto seed-om namiesto `randombytes_buf()`; každý beh je individuálne reprodukovateľný opakovaním s rovnakým seed-om.
- Čerstvý `ExperimentContext` (vrátane SQLite databázy) pre každú kombináciu (profil, step, run).

- CLI prepínače `-seed=N`, `-warmup=N`, `-baseline=N`, `-runs=N`, `-e2e=URL`, `-profile=P` umožňujú reprodukovateľné experimenty a E2E testovanie cez HTTP.

Vzťah harnessu k produkčnému systému Harness je *kontrolovaný experimentálny mini-systém*, nie plná replika produkčného systému. Zdieľa s produkčným kódom kryptografickú vrstvu (`crypto_lib`), deriváciu kľúčov (`key_manager`), rozhodovaciu logiku (`DecisionEngine`), anti-replay mechanizmus (`ReplayWindow`) a anomálny detektor (`ProtocolAnomalyDetector`). Zámerne sú vynechané vrstvy, ktoré nie sú predmetom merania:

- **HTTP transport a HMAC autentifikácia požiadavky** — harness volá `DecisionEngine::handleFrameBytes()` priamo, bez sieťového overhead-u. Tým sa izoluje latencia kryptografických a rozhodovacích operácií od variability sieťového zásobníka.
- **Kontrola čerstvosti (`maxSkewMs = 0`)** — overenie časovej pečiatky je v harness vypnuté, aby sa predišlo kontaminácii výsledkov efektmi časovej synchronizácie. Experimenty sa zameriavajú na nonce, replay a detektor, nie na hodiny.
- **Perzistentný auditný log** — harness používa in-memory audit sink namiesto HMAC hash chain s SQLite perzistenciou. Auditná integrita je testovaná samostatne v unit testoch; v experimentoch by perzistentný zápis pridával I/O latenciu nesúvisiacu s meranými mechanizmami.

Šesť konfiguračných profilov (A1-R0 až A2-R2) je definovaných programaticky vo funkcii `allProfiles()`, nie načítaných zo súboru YAML. Hodnoty parametrov (algoritmus, režim nonce, stav detektora) sú ekvivalentné produkčným YAML konfiguráciám, ale priama definícia v kóde eliminuje závislosť na externom súbore a zaisťuje, že profily sa nemôžu meniť medzi behmi.

3.3.3 Popis útokových scenárov

S1: Replay útok Scenár zachytí validný frame v polovici baseline fázy a opakovane ho posíla v trial fáze. Modeluje útočníka, ktorý zachytil sieťovú komunikáciu a reprodukuje ju bez modifikácie — štandardný replay útok podľa klasifikácie NIST[17].

Očakávané správanie: Replay window odchytil duplikát sekvenčného čísla. R2 navyše zakaranténuje čítačku s anomáliou `seq_reuse`.

S2: Manipulácia poľa v hlavičke (AAD tampering) Scenár vytvorí validný frame a následne zmení `door_id` v binárnej serializácii hlavičky bez opätovného šifrovania payloadu. Overuje AAD binding: cieľom je zachytiť nekonzistentnosť medzi hlavičkou a payloadom na úrovni kryptografickej väzby — nie na úrovni sieťovej vrstvy alebo aplikačného smerovania.

Konfigurácia: V harness sa podvrhnutý `door_id` (pôvodný +999) explicitne registruje ako povolené dvere pre danú čítačku (`allowDoorForReader`). Tým sa izoluje kryptografická detekcia manipulácie od autorizačnej politiky — bez tohto kroku by frame bol zamietnutý už na úrovni reader-door bindingu (`unknown_reader`), čo by zakrylo správanie AEAD/AAD vrstvy.

Očakávané správanie: Keďže `door_id` je súčasťou AAD, zmena spôsobí zlyhanie AEAD overenia. R0/R1 vracajú `decrypt_failed`. R2 detekuje problém skôr — zmena `door_id` mení kontext pre deterministický nonce, takže overenie nonce zlyhá (`nonce_mismatch`) ešte pred pokusom o dešifrovanie.

S3a: Overenie nonce enforcement politiky Scenár injektuje chybu do generovania nonce: trieda `FixedNonceGenerator` naplňa celý nonce bajtom 0xAA, čím produkuje rovnaký nonce pre každý frame bez ohľadu na kontext a sekvenciu. Cieľom je overiť, či a ako jednotlivé nonce režimy reagujú na porušenie deterministického nonce invariantu.

Očakávané správanie: R0/R1 nemajú verifikáciu nonce — engine použije nonce z

hlavičky frame na dešifrovanie bez ďalšej kontroly. AEAD dešifrovanie uspeje (kľúč je správny, nonce je len vstupný parameter). R2 rekonštruuje očakávaný nonce z rovnakého kľúča a kontextu a porovná ho s hodnotou v hlavičke; pri nesúlade vyvolá `nonce_mismatch` s okamžitou karanténou.

Prečo je tento test dôležitý: S3a je jediný scenár, ktorý priamo testuje **nonce enforcement** — odlišuje R0/R1 od R2 práve v kontexte nonce politiky (S4 odlišuje R0/R1 od R2 cez seq rollback detekciu, nie nonce). V aktuálnej architektúre s oddeleným DecisionService (port 8081) AEAD frame cestuje cez HTTP kanál, kde generátor nonce (frontend) a verifikátor (DecisionService) sú oddelené služby. Vynucovaná nonce politika v R2 zabezpečuje, že akákoľvek odchýlka — či už spôsobená chybou, konfiguračným nesúlodom alebo zásahom na kanáli — je detekovaná a zariadenie zakaranténované[29].

S3b: Izolácia kľúčov medzi čítačkami (HKDF key isolation)

Komponentový validačný scenár overuje, že frame vytvorený v kľúčovom kontexte jednej čítačky nemožno úspešne overiť v kontexte inej čítačky. Nejde o simuláciu aktuálneho produkčného HTTP toku zariadenia (čítačka v produkcii nekonštruuje AEAD frame), ale o overenie izolačných vlastností HKDF per-reader derivácie v AEAD vrstve.

HKDF[19] derivuje unikátny kľúč pre každú čítačku:

$$K_{\text{aead}}^{(r)} = \text{HKDF}(\text{master}, \text{salt} = \text{reader_id} \parallel \text{key_version}, \text{info} = \text{"access-aead-v1"}) \quad (3.4)$$

Postup: Harness zašifruje frame kľúčom čítačky 2 ($K_{\text{aead}}^{(2)}$), ale hlavičku frame podvrhne tak, aby obsahovala `reader_id=1`. Engine parsuje hlavičku, derivuje kľúč pre čítačku 1 ($K_{\text{aead}}^{(1)}$) a pokúsi sa o AEAD dešifrovanie. Zlyhanie je dvojnásobné: kľúč aj AAD sú nekonzistentné so šifrovaným obsahom.

Očakávané správanie: Všetky profily odmietajú 100% útočných frame (`decrypt_failed`). R2 navyše detekuje nonce anomáliu okamžite (nonce bol derivovaný z kontextu čítačky 2) — `nonce_mismatch` vedie ku karanténe od prvého

frame. Kľúčový záver: HKDF per-reader derivácia poskytuje kryptografickú izoláciu (compartmentalization) — kompromitácia jednej čítačky neohrozí ostatné.

S4: Rollback sekvenčného čísla Scenár posiela frame so sekvenčnými číslami z “rollback zóny” — hodnoty, ktoré sú nižšie ako maximálne videné číslo, ale nie sú príliš staré pre replay window a neboli nikdy odoslané.

Rollback zóna je definovaná:

$$\text{rollback zone} = [\text{maxSeen} - W + 1, \text{maxSeen} - \delta) \quad (3.5)$$

kde $W = 256$ je veľkosť replay window a $\delta = 100$ je rollback threshold.

Očakávané správanie: Replay window tieto sekvencie neodmietne, pretože nie sú “too old” ($> \text{maxSeen} - W$) ani v množine videných. R0/R1 teda čiastočne prepustia útok. R2 detekuje rollback cez podmienku $\text{seq} + \delta < \text{maxSeenSeq}$ a okamžite zakaranténuje čítačku.

Matematika úspešnosti pre R0/R1: Z 155 slotov v rollback zóne sa 99 prekrýva s baseline sekvenciami v `_seen` sade (60000–60098). Reálne prejde len 56 unikátnych sekvencií (59944–59999). Pri $N = 500$: $56/500 = 11.2\%$.

S5: Opakované neplatné ciphertexty (AEAD tag verification) Scenár vytvorí validný frame a následne poškodí ciphertext deterministickou bitovou inverziou (XOR 0xFF). Overuje, či AEAD tag verifikácia zachytí poškodenú šifrovanú časť frame — formálne ide o test odolnosti použitia AEAD voči forgery pokusom[23]. Nejde o modelovanie externého útočníka, ktorý posiela ciphertext po sieti; frame je interná štruktúra systému. Scenár testuje správanie overenia tagu pri chybnom alebo zámernom poškodení v rámci spracovacej cesty.

Očakávané správanie: AEAD overenie zlyhá pre všetky profily (`decrypt_failed`). R2 navyše sleduje sériu zlyhaní a po dosiahnutí limitu 5 zakaranténuje čítačku s anomáliou `tag_fail_streak`.

S6: Throughput benchmark Čisto výkonnostný scenár. Posiela len validné frame (detektor sa neaktivuje). Meria latenciu spracovania a priepustnosť.

- Warmup: 1000 frame, baseline: 0, trial: 1000/5000/10000/50000 frame.
- 5 behov s odlišnými seed-mi (42–46) na posúdenie variability a robustnosti výsledkov naprieč behmi.
- Fáza trial označená ako “measure” (nie “attack”).

S7: Nonce tamper (MITM na kanáli) Scenár simuluje útočníka, ktorý zachytí validný AEAD frame na komunikačnom kanáli medzi frontend službou a DecisionService a modifikuje nonce v hlavičke (XOR 0xFF na všetkých 24 bajtoch). Frame bol pôvodne platný — kľúč, payload aj AAD sú správne — ale nonce je zmenený v transite.

Očakávané správanie: Zmena nonce spôsobí zmenu keystream-u pri AEAD dešifrovaní, čo vedie k zlyhaniu Poly1305 tag verifikácie. Všetky profily odmietajú frame (`decrypt_failed`). R2 navyše detekuje odchýlku nonce od očakávanej deterministickej hodnoty (`nonce_mismatch`) ešte pred pokusom o dešifrovanie a okamžite zakaranténuje zariadenie.

Rozdiel oproti S3a: S3a injektuje chybu do generátora nonce (fault model — frame je od začiatku vytvorený s nesprávnym nonce). S7 začína s platným frame a modifikuje ho v transite (network MITM model). Architektúra s oddeleným DecisionService na porte 8081 robí tento scenár realistickým — frame skutočne cestuje cez HTTP kanál.

3.3.4 Metodika zberu a spracovania dát

Zber dát Každý scenár produkuje CSV súbor s jedným riadkom na frame. Celkový objem dát pri 5 behoch: ~2,835,000 riadkov. Pre každý frame sa zaznamenáva:

- konfiguračný profil a jeho parametre,

- fáza (baseline/attack/measure), počet útočných frame a index behu,
- rozhodnutie (**allowed**), dôvod (**reason**), stav karantény,
- typ anomálie (ak bola detekovaná detektorom),
- latencia spracovania v nanosekundách.

Odvodzované metriky Z primárnych dát sa počítajú nasledujúce metriky:

Attack success rate: Podiel útočných frame, ktoré boli povolené (**allowed** = true), agregovaný cez behy (priemer $\pm$ smerodajná odchýlka).

False reject rate: Podiel baseline frame, ktoré boli zamietnuté (meria falošné odmietnutia validných požiadaviek).

Time to quarantine: Index prvého frame, kde bol čítač zakaranténovaný (len R2 profily).

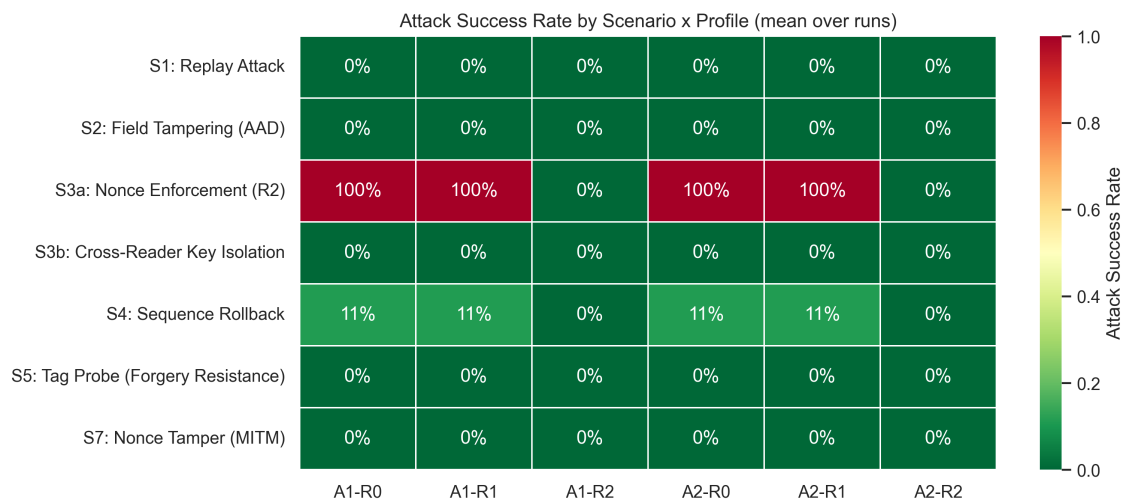
Throughput: $\text{FPS} = \frac{\text{počet frame}}{\text{latencia}_{\text{ns}}/10^9}$, počítaný per (profil, N , beh).

Latencia: Percentily p50, p95, p99 v μs .

Štatistická robustnosť Každý beh r ($r = 0 \dots 4$) používa seed $42+r$, takže nonce hodnoty sa líšia medzi behmi. Pre **bezpečnostné metriky** (S1–S5) 5 behov slúži ako kontrola reprodukovateľnosti: keďže ochranné mechanizmy (AEAD overenie, replay window, detektor) sú invariantné voči konkrétnym hodnotám nonce, výsledok je binárny a zhodný naprieč všetkými behmi. Pásma v grafoch majú nulovú šírku — nie ako výsledok štatistickej inferencie, ale ako vizuálne potvrdenie konzistentnosti. Pre **latenciu a priepustnosť** (S6) sú behy skutočným zdrojom variability (OS a HW šum) a štandardná odchýlka má bežný štatistický výklad; dosahuje 1–4 % priemeru.

3.3.5 Výsledky: Úspešnosť útokov

Celkový prehľad Obrázok 3–2 zobrazuje úspešnosť útokov pre všetky kombinácie scenár × profil pri maximálnom počte útočných frame ($N = 500$, resp. $N = 50,000$ pre S6).



Obrázok 3–2 Úspešnosť útokov podľa scenára a profilu (priemer cez 5 behov). Zelené bunky: útok zablokovaný (0%). Červené bunky: útok úspešný.

Tabuľka 3–7 Úspešnosť útoku a reakcia detektora podľa scenára a nonce stratégie

Scenár	R0/R1		R2		
	SR	Reakcia	SR	Reakcia	Anomália
S1: Replay	0%	replay	0%	quarantined	seq_reuse
S2: AAD Tamper	0%	decrypt_failed	0%	quarantined	nonce_mismatch
S3a: Nonce Enf.	100%	ok	0%	quarantined	nonce_mismatch
S3b: Cross-Reader	0%	decrypt_failed	0%	quarantined	nonce_mismatch
S4: Seq Rollback	11%	replay	0%	quarantined	seq_rollback
S5: Tag Probe	0%	decrypt_failed	0%	quarantined	tag_fail_streak
S7: Nonce Tamper	0%	decrypt_failed	0%	quarantined	nonce_mismatch

SR = úspešnosť útoku (priemer cez 5 behov). A1 a A2 vykazujú identické výsledky — tabuľka agreguje podľa nonce stratégie (R0/R1/R2).

Z tabuľky 3–7 a obrázka 3–2 vyplýva jasné delenie:

- **S1, S2, S5** — pri žiadnom profile nedošlo k prijatiu škodlivého frame (0% úspešnosť). Ochranu zabezpečuje replay window (S1), AAD binding (S2) a AEAD tag verifikácia (S5).
- **S3a** — nonce enforcement: R0/R1 prepustia frame s chybným nonce (100%); R2 odchýlku zachytí a zakaranténuje (0%).
- **S3b** — cross-reader key isolation: žiadny profil neprijal frame z kontextu inej čítačky (0%); R2 karanténuje od prvého frame.
- **S4** — R0/R1 čiastočne zraniteľné (11%), R2 blokuje (0%).
- **S7** — nonce tamper (MITM): zmena nonce v transite spôsobí zmenu keystream-u a zlyhanie Poly1305 tag verifikácie. Všetky profily odmietajú frame (0%); R2 navyše detekuje odchýlku nonce ešte pred dešifrovaním (`nonce_mismatch`) a zakaranténuje zariadenie.

Závislosť od počtu útočných frame Pre väčšinu scenárov je úspešnosť útoku konštantná bez ohľadu na počet útočných frame N — buď 0% (S1, S2, S3b, S5, S7 pre všetky profily; S3a, S4 pre R2), alebo 100% (S3a pre R0/R1). Jedinou výnimkou je S4 (Sequence Rollback): úspešnosť R0/R1 klesá z $\sim 100\%$ pri $N = 50$ na $\sim 11\%$ pri $N = 500$, pretože rollback zóna obsahuje len 56 unikátnych sekvencií — po ich vyčerpaní replay window odchyťava opakované hodnoty. R2 profily vykazujú konštantnú nulovú úspešnosť pre všetky N . Výsledky sú konzistentné naprieč všetkými 5 behmi (nulový rozptyl).

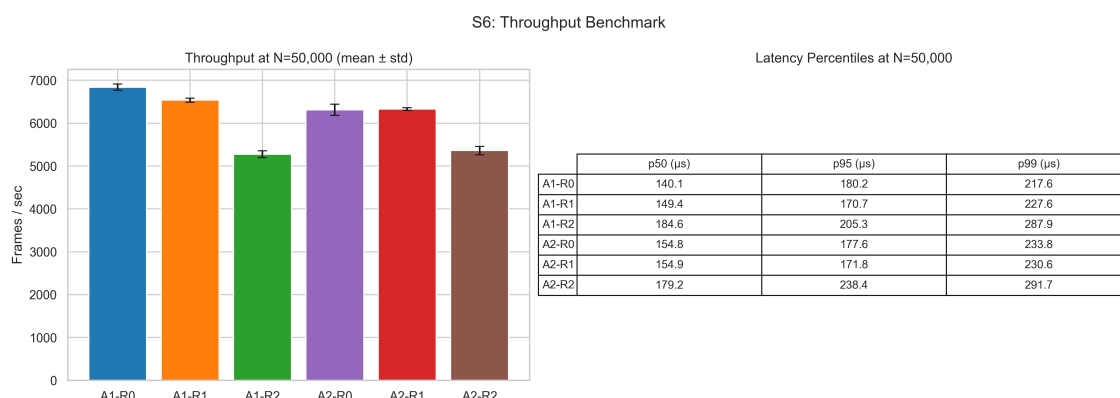
Rýchlosť karantény Pre scenáre S1–S4 a S7 nastáva karanténa R2 profilov na prvom útočnom frame (`frame_idx = 0`). Pre S5 (tag probe) nastáva na `frame_idx = 4`, čo zodpovedá nakonfigurovanému limitu `tag_fail_streak_limit = 5` (prvé 4 zlyhania sú tolerované; 5. zlyhanie aktivuje karanténu). Hodnoty A1–R2

a A2–R2 sú identické, čo potvrdzuje, že rýchlosť detekcie závisí od konfigurácie detektora, nie od zvoleného šifrovacieho algoritmu.

3.3.6 Výsledky: Baseline korektnosť

Baseline fáza (200 validných frame pred každým útokom) vykazuje **nulovú false reject rate** (0,00%) naprieč všetkými 7 scenármi a 6 profilmi. Žiadny validný frame nebol zamietnutý. Toto je nevyhnutná podmienka pre vierohodnosť bezpečnostných výsledkov — ak by baseline vykazoval falošné odmietnutia, nebolo by možné jednoznačne interpretovať úspešnosť útokov.

3.3.7 Výsledky: Latencia a priepustnosť



Obrázok 3–3 Priepustnosť pri $N = 50,000$ frame (priemer $\pm$ std cez 5 behov) a percentily latencie.

Throughput benchmark (S6) Obrázok 3–3 zobrazuje distribúciu priepustnosti pre každý profil. Všetky profily dosahujú priepustnosť nad 5,000 frame/s, čo je rádovo viac ako reálna záťaž fyzického prístupového systému (<10 udalostí/s). Mediánová latencia je 140–185 μ s.

Porovnanie algoritmov: A2 (XChaCha20) dosahuje mierne vyššiu priepustnosť ako A1 (ChaCha20), čo je prekvapivý výsledok. Príčinou je pravdepodobne optimalizácia libsodium pre XChaCha20 cestu.

Tabuľka 3–8 Priepustnosť a latencia pri N=50,000 frame

Profil	FPS (priemer)	FPS (std)	p50 (μs)	p95 (μs)	p99 (μs)
A1–R0	6,840	72	140.1	180.2	217.6
A1–R1	6,537	49	149.4	170.7	227.6
A1–R2	5,276	80	184.6	205.3	287.9
A2–R0	6,312	130	154.8	177.6	233.8
A2–R1	6,326	32	154.9	171.8	230.6
A2–R2	5,361	97	179.2	238.4	291.7

Overhead detektora: Porovnanie R1 a R2 umožňuje izolovať overhead anomálneho detektora. Rozdiel je $\sim 15\text{--}19\%$ v závislosti od algoritmu (A1–R1: 6 537 fps vs. A1–R2: 5 276 fps, t.j. $-19,3\%$; A2–R1: 6 326 fps vs. A2–R2: 5 361 fps, t.j. $-15,3\%$). Overhead pochádza z:

- výpočtu očakávaného nonce (HKDF derivácia + HMAC),
- porovnania nonce v konštantnom čase (`sodium_memcmp`),
- aktualizácie stavu detektora (rollback check, tag streak).

Overhead detektora na baseline frame Porovnanie R1 a R2 na validných baseline frame (S1–S7) izoluje overhead detektora: medián latencie R2 je o $\sim 25\text{--}35 \mu s$ vyšší (A1: $\sim 35 \mu s$, A2: $\sim 24 \mu s$), čo zodpovedá dodatočnej HKDF derivácii nonce kľúča, HMAC výpočtu a aktualizácii stavu detektora. Distribúcia latencie podľa scenárov ukazuje aj rozdiel medzi baseline a attack fázou: pri S1 (replay) je attack latencia nižšia, pretože replay window odmietne frame skôr bez pokusu o dešifrovanie; pri S2 a S5 je attack latencia vyššia pre R2, pretože detektor vykonáva dodatočné kontroly pred zamietnutím.

3.3.8 Diskusia a interpretácia výsledkov

Experimentálna analýza potvrdila šesť hlavných zistení:

1. **T1 — Nonce enforcement a HKDF izolácia.** S3a: R0/R1 prepustia frame s chybným nonce (100%), R2 zakaranténuje (0%). S3b: žiadny profil neprijal cross-reader frame (HKDF izolácia). Nonce enforcement diferencuje nonce režimy; HKDF izolácia zabezpečuje compartmentalization nezávisle od nonce režimu[29, 19].
2. **T2 — ProtocolAnomalyDetector.** R2 dosiahol 0% úspešnosť naprieč S1–S7. V S3a a S4 bol jediným mechanizmom, ktorý scenár zastavil; v S3b a S5 pridáva operačnú eskaláciu (karanténu) k existujúcemu kryptografickému odmietnutiu.
3. **T3 — Replay window a detektor sú komplementárne.** S4 ukazuje, že replay window prepúšťa sekvencie v rollback zóne (11%); detektor ich zachytí cez podmienku $\text{seq} + \delta < \text{maxSeenSeq}$. Replay window odchyťava duplikáty; detektor štrukturálne anomálie[16, 25].
4. **T4 — AAD binding.** S2: manipulácia `door_id` detekovaná všetkými profilmi. R0/R1 cez AEAD tag failure; R2 cez `nonce_mismatch` (skoršia detekcia). AAD binding funguje nezávisle od nonce režimu.
5. **T5 — Defense-in-depth cez karanténu.** R2 nelen odmieta frame ale izoluje celé zariadenie. Realizácia princípu defense-in-depth[25]: vrstvy eskalujú reakciu.
6. **T6 — Overhead detektora.** A1: R1 (6,537 fps) $\rightarrow$ R2 (5,276 fps): -19.3% ; A2: R1 (6,326 fps) $\rightarrow$ R2 (5,361 fps): -15.3% . Pri reálnej záťaži (<10 udalostí/s) rádovo nepodstatný[31].

3.3.9 Porovnanie s minimalistickými implementáciami

Pre kontext porovnáваме s hypotetickými architektúrami: B0 (bez kryptografie), B1 (HTTPS + API kľúč, bez AEAD/anti-replay/audit), A1-R0 (náš základný profil) a A1-R2/A2-R2 (plný profil s detektorom).

Tabuľka 3–9 Odolnosť voči útočným scenárom podľa úrovne implementácie

Scenár / útok	B0	B1	A1-R0	A1-R2 / A2-R2
S1 Replay	×	×	✓	✓ + karanténa
S2 Tampering (AAD)	×	častočne	✓	✓ (pred dekr.)
S3a Nonce enforcement	n/a	n/a	×	✓
S3b Cross-reader	×	×	✓	✓ + karanténa
S4 Seq rollback	×	×	častočne	✓
S5 Tag probe	×	×	✓	✓ + karanténa
S7 Nonce tamper	×	×	✓	✓ + karanténa
Manipulácia auditu	×	×	✓	✓
Identita karty (DB)	×	×	✓ (HMAC)	✓

Prechod z B0 na A1-R0 je kvalitatívny skok; R2 uzatvára zvyšné medzery (S3a, S4) a pridáva karanténu[25, 16].

3.3.10 Zhrnutie experimentov

Tabuľka 3–7 a obrázok 3–2 poskytujú súhrnný prehľad výsledkov. Experimentálna analýza siedmich scenárov naprieč šiestimi konfiguračnými profilmi potvrdila nasledujúce zistenia:

1. **Nulová false reject rate:** Žiadny validný frame nebol zamietnutý žiadnym profilom.
2. **Nonce enforcement a HKDF izolácia (S3a/S3b):** S3a ukazuje, že R2 ako jediný vynucuje nonce politiku — poistka pre budúce škálovanie. S3b potvrdzuje, že HKDF per-reader derivácia izoluje kľúčový materiál medzi čítačkami naprieč všetkými profilmi.

-
3. **R2 zablokoval všetky testované scenáre:** 0% úspešnosť naprieč S1–S7 v rámci tejto experimentálnej sady (vrátane S3a, ktorý je komponentovou validáciou nonce mechanizmu, nie simuláciou externého útoku).
 4. **Replay window a detektor sú komplementárne:** S4 odhaľuje štruktúrnu medzeru replay window, ktorú detektor uzatvára.
 5. **AAD binding je fundamentálna ochrana:** Funguje nezávisle od nonce režimu.
 6. **Karanténa pridáva proaktívnu izoláciu zariadení:** Kvalitatívne odlišná od per-frame odmietnutia.
 7. **Overhead detektora (~15–19%):** Pri reálnej záťaži prístupového systému (<10 udalostí/s) rádovo nepodstatný; pri vyššej záťaži by mal byť zohľadnený.
 8. **Výsledky sú reprodukovateľné:** Všetkých 5 behov (seedy 42–46) poskytlo zhodné výsledky attack success rate. Bezpečnostné mechanizmy sú invariantné voči hodnotám nonce — 5 behov tu neplní úlohu štatistického vzorku, ale overenia konzistencie výsledkov.

Tieto výsledky potvrdujú pôvodnú hypotézu práce: kombinácia AEAD s AAD binding, deterministickým nonce a runtime anomálnou detekciou (profil R2) **materiálne zvyšuje odolnosť** prístupového systému v porovnaní so základnými konfiguráciami, a to pri prijateľnom výkonnostnom overeade.

4 Záver (zhodnotenie riešenia)

Práca sa zaoberala návrhom, implementáciou a experimentálnym hodnotením bezpečnostných mechanizmov pre systém kontroly fyzického prístupu na báze RFID/NFC. Výsledkom je funkčný prototyp pozostávajúci z hardvérovej čítačky (ESP32 + RC522) a dvoch nezávislých serverových služieb v jazyku C++20 — frontend služba (spracovanie požiadaviek, HMAC autentifikácia, konštrukcia AEAD frame) a DecisionService (dešifrovanie, rozhodovacia logika, anomálna detekcia) — s lokálnym úložiskom SQLite. Systém implementuje viacvrstvovú ochranu: HMAC-SHA-256 autentifikáciu hardvérových požiadaviek, AEAD frame (ChaCha20-Poly1305 / XChaCha20-Poly1305) s AAD binding, HKDF per-reader deriváciu kľúčov, anti-replay sliding window, anomálny detektor s karanténou a tamper-evident audit log. Architektúra s oddeleným DecisionService umožňuje nezávislé testovanie bezpečnostných vlastností AEAD vrstvy a zároveň robí scenár S7 (nonce tamper na komunikačnom kanáli) realistickým.

Okrem samotného prototypu práca prináša reprodukovateľný experimentálny rámec: automatizovaný harness izoluje útočný vstup, cieľové primitívy a metriky do samostatných komponentov, Python pipeline agreguje výstupy do CSV a generuje grafy aj LaTeX tabuľky použité v texte. Porovnanie profilov prebieha nad spoločnou produkčnou kódovou základňou so zdieľaným AEAD enginom, čím je zabezpečená neutralita vyhodnotenia — žiadny profil nemá implementačnú výhodu nad ostatnými.

Prototyp je kontajnerizovateľný pomocou priloženého `Dockerfile`; modulárna štruktúra serverovej aplikácie umožňuje výmenu jednotlivých vrstiev bez dopadu na ostatné (napr. nahradenie SQLite plnohodnotnou SQL databázou alebo pridanie ďalších AEAD algoritmov). Použitie výlučne permissívnych open-source závislostí odstraňuje licenčné prekážky pre ďalšie rozšírenia.

Experimentálna analýza šiestich profilov (tabuľka 4–1) ($A1/A2 \times R0/R1/R2$) v siedmich scenároch potvrdila hlavné zistenia:

Tabuľka 4 – 1 Súhrn hlavných experimentálnych zistení

Téza	Zistenie
T1	R2 ako jediný vynucuje nonce politiku (S3a: R0/R1 100% úspešnosť vs. R2 0%). HKDF per-reader derivácia zabráňuje cross-reader útokom (S3b: 0% naprieč všetkými profilmi).
T2	<code>ProtocolAnomalyDetector</code> (profily R2) dosiahol 0% úspešnosť naprieč všetkými testovanými scenármi (S1–S7). V S3a a S4 bol jediným mechanizmom, ktorý scenár zastavil; v S7 detekoval nonce tamper ešte pred pokusom o dešifrovanie.
T3	Replay window a anomálna detekcia sú komplementárne. S4: 11% pre R0/R1 $\rightarrow$ 0% pre R2.
T4	AAD binding poskytuje ochranu nezávisle od režimu nonce (S2: 0% naprieč všetkými profilmi). S7 potvrdzuje, že zmena nonce v transite vedie k zlyhaniu AEAD tag verifikácie aj bez detektora.
T5	Karanténa mení reakciu z per-frame odmietnutia na izoláciu zariadenia (defense-in-depth).
T6	Overhead detektora ($\sim 15\text{--}19\%$, $\sim 25\text{--}35\ \mu\text{s}$ na frame v závislosti od AEAD algoritmu) je rádovo nepodstatný pre reálnu prevádzku (< 10 udalostí/s), hoci nie je zanedbateľný.

Obmedzenia a budúca práca

Práca vedome zjednodušuje niektoré aspekty produkčného nasadenia: karta slúži ako identifikátor, TLS nie je povinné a správa tajomstiev je pragmatická (hex súbor, nie HSM). Tieto obmedzenia vymedzujú rámec pre interpretáciu experimentov.

Smery budúcej práce: mTLS[28] pre transport, migrácia na kryptografické smart karty (napr. MIFARE DESFire[10]) a integrácia s centralizovanou správou tajomstiev (HSM/vault)[3].

Zoznam použitej literatúry

- [1] Adam Arciszewski. draft-irtf-cfrg-xchacha-01: XChaCha: eXtended-nonce ChaCha and AEAD_XChaCha20_Poly1305. IETF Datatracker (Internet-Draft), 2019. Available from: <https://datatracker.ietf.org/doc/html/draft-irtf-cfrg-xchacha-01>.
- [2] ASSA ABLOY. Aperio wireless lock technology. ASSA ABLOY, 2023. Available from: <https://www.assaabloy.com/group/solutions/aperio>.
- [3] Elaine Barker. NIST SP 800-57 Part 1 Rev. 5: Recommendation for Key Management – Part 1: General. NIST, 2020. Available from: <https://csrc.nist.gov/pubs/sp/800/57/pt1/r5/final>.
- [4] Mihir Bellare, Alexandra Boldyreva, and Jesse O’Neill. Deterministic and efficiently searchable encryption. In *Advances in Cryptology — CRYPTO 2007*, volume 4622 of *Lecture Notes in Computer Science*, pages 535–552, 2007.
- [5] Daniel J. Bernstein. The salsa20 family of stream ciphers. In *New Stream Cipher Designs*, volume 4986 of *Lecture Notes in Computer Science*, pages 84–97. Springer, 2008.
- [6] Frank Denis. The Sodium cryptography library (libsodium). Online documentation, 2024. Available from: <https://doc.libsodium.org/>.
- [7] Espressif Systems. ESP32 Technical Reference Manual, Version 5.3. Espressif Systems, 2024. Available from: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf.
- [8] Hildegard Ferraiolo, Ketan Mehta, Nabil Ghadiali, Jason Mohler, Vincent Johnson, and Steven Brady. Guidelines for PACS — physical access control systems. Technical Report SP 800-116 Rev. 1, NIST, 2018. Available from: <https://doi.org/10.6028/NIST.SP.800-116r1>.

-
- [9] Zac Franken. Wiegand protocol: The elephant in the room. DEF CON 21, 2013. Presentation on Wiegand vulnerabilities in physical access control.
 - [10] Flavio D. Garcia, Gerhard de Koning Gans, Ruben Muijers, Peter van Rossum, Roel Verdult, Ronny Wichers Schreur, and Bart Jacobs. Dismantling MIFARE classic. In *Proceedings of the 13th European Symposium on Research in Computer Security (ESORICS)*, volume 5283 of *Lecture Notes in Computer Science*, pages 97–114, 2008.
 - [11] Google. GoogleTest — Google Testing and Mocking Framework. GitHub, 2024. Available from: <https://github.com/google/googletest>.
 - [12] Shawn Hernan, Scott Lambert, Tomasz Ostwald, and Adam Shostack. Threat modeling — uncover security design flaws using the STRIDE approach. *MSDN Magazine*, November 2006.
 - [13] HID Global. iCLASS SE platform — secure identity solutions. HID Global, 2023. Available from: <https://www.hidglobal.com/solutions/iclass-se>.
 - [14] International Organization for Standardization. ISO/IEC 14443:2018 Identification cards — Contactless integrated circuit cards — Proximity cards. ISO, 2018.
 - [15] International Organization for Standardization. ISO/IEC 27001:2022 Information security, cybersecurity and privacy protection — Information security management systems — Requirements. ISO, 2022.
 - [16] Joint Task Force. NIST SP 800-53 Rev. 5: Security and Privacy Controls for Information Systems and Organizations. NIST, 2020. Available from: <https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final>.
 - [17] Theodore Karygiannis, Barry Eydt, Gregory Barber, Lynn Bunn, and Ted Phillips. NIST SP 800-98: Guidelines for Securing Radio Frequency

-
- Identification (RFID) Systems. NIST, 2007. Available from: <https://csrc.nist.gov/pubs/sp/800/98/final>.
- [18] Karen Kent and Murugiah Souppaya. NIST SP 800-92: Guide to Computer Security Log Management. NIST, 2006. Available from: <https://csrc.nist.gov/pubs/sp/800/92/final>.
- [19] Hugo Krawczyk and Phil Eronen. RFC 5869: HMAC-based Extract-and-Expand Key Derivation Function (HKDF). IETF, 2010. Available from: <https://datatracker.ietf.org/doc/html/rfc5869>.
- [20] Hugo Krawczyk, Mihir Bellare, and Ran Canetti. RFC 2104: HMAC: Keyed-Hashing for Message Authentication. RFC Editor, 1997. Available from: <https://www.rfc-editor.org/rfc/rfc2104.html>.
- [21] libsodium contributors. XChaCha20-Poly1305 construction – libsodium documentation. Online documentation, 2025. Available from: https://libsodium.gitbook.io/doc/secret-key_cryptography/aead/chacha20-poly1305/xchacha20-poly1305_construction.
- [22] LLVM Project. Clang-Tidy — Extra Clang Tools Documentation. LLVM Project, 2024. Available from: <https://clang.llvm.org/extra/clang-tidy/>.
- [23] David McGrew. RFC 5116: An Interface and Algorithms for Authenticated Encryption. RFC Editor, 2008. Available from: <https://www.rfc-editor.org/rfc/rfc5116.html>.
- [24] National Institute of Standards and Technology. FIPS 198-1: The Keyed-Hash Message Authentication Code (HMAC). NIST, 2008. Available from: <https://csrc.nist.gov/pubs/fips/198-1/final>.
- [25] National Security Agency. Defense in Depth: A practical strategy for achieving Information Assurance in today’s highly networked environments.
-

-
- NSA/IAD, 2012. Available from: <https://apps.nsa.gov/iaarchive/library/ia-guidance/archive/defense-in-depth.cfm>.
- [26] Yoav Nir. RFC 8439: ChaCha20 and Poly1305 for IETF Protocols. RFC Editor, 2018. Available from: <https://www.rfc-editor.org/rfc/rfc8439.html>.
- [27] Private Internet Access and NCC Group. Libsodium Security Assessment. Private Internet Access, 2017. Available from: <https://www.privateinternetaccess.com/blog/libodium-v1-0-12-and-target-v1-0-13-security-assessment/>.
- [28] Eric Rescorla. RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3. RFC Editor, 2018. Available from: <https://www.rfc-editor.org/rfc/rfc8446.html>.
- [29] Phillip Rogaway and Thomas Shrimpton. A provable-security treatment of the key-wrap problem. In *Advances in Cryptology — EUROCRYPT 2006*, volume 4004 of *Lecture Notes in Computer Science*, pages 373–390, 2006.
- [30] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.
- [31] Karen Scarfone, Wayne Jansen, and Miles Tracy. NIST SP 800-94: Guide to Intrusion Detection and Prevention Systems (IDPS). NIST, 2007. Available from: <https://csrc.nist.gov/pubs/sp/800/94/final>.
- [32] Bruce Schneier and John Kelsey. Secure audit logs to support computer forensics. *ACM Transactions on Information and System Security*, 2(2):159–176, 1999. DOI: 10.1145/317087.317089.
- [33] Security Industry Association. OSDP — open supervised device protocol. SIA Standard, 2020. OSDP v2, IEC 60839-11-5. Available from: <https://www.securityindustry.org/osdp/>.
-

-
- [34] Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov. AddressSanitizer: A fast address sanity checker. In *Proceedings of the 2012 USENIX Annual Technical Conference*, pages 309–318, 2012.
- [35] Yaron Sheffer, Deirdre Connolly Benjamin, Martin Thomson, et al. RFC 9325: Recommendations for Secure Use of Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS). RFC Editor, 2022. Available from: <https://www.rfc-editor.org/rfc/rfc9325.html>.
- [36] Roel Verdult, Flavio D. Garcia, and Baris Ege. Dismantling megamos crypto: Wirelessly lockpicking a vehicle immobilizer. In *Proceedings of the 24th USENIX Security Symposium*, pages 703–718, 2015.
- [37] Ömer Şiar Baysal. ESP-RFID: Access control system using ESP8266/ESP32 and MFRC522. GitHub, 2023. Available from: <https://github.com/esprfid/esp-rfid>.

Zoznam príloh

Príloha A Systémová príručka

Príloha B Používateľská príručka

Príloha C Zdrojový kód prototypu

Príloha D Výstupy experimentov (CSV súbory)

Príloha A

Systémová príručka popisuje architektúru prototypu, moduly serverovej aplikácie, schému úložiska, konfiguráciu, postup prekladu a nasadenia, závislosti a celkové zhodnotenie riešenia. Dokument je priložený v samostatnom PDF súbore a v zdrojovom $\text{\LaTeX}$ tvare.

Príloha B

Používateľská príručka popisuje funkciu programu, súpis obsahu dodávky, inštaláciu, spustenie, webové administrátorské rozhranie, vstupné a výstupné súbory, obmedzenia a chybové hlásenia. Dokument je priložený v samostatnom PDF súbore a v zdrojovom $\text{\LaTeX}$ tvare.

Príloha C

Zdrojový kód prototypu obsahuje kompletnú implementáciu: produkčný serverový kód (moduly `crypto_lib`, `protocol_lib`, `key_manager`, `access_core`, `access_decision`, `access_storage`, `access_admin`), firmvér čítačky ESP32, experimentálny harness, analytický Python pipeline, konfiguračné súbory (YAML profily), Dockerfile a sadu automatizovaných testov (Google Test).

Príloha D

CSV výstupy experimentov obsahujú per-frame záznamy pre všetkých šesť konfiguračných profilov ($A1/A2 \times R0/R1/R2$) vo všetkých siedmich útočných scenároch (S1–S7). Celkový objem dát presahuje 2,8 milióna meraní a slúži ako podklad pre súhrnné tabuľky a grafy prezentované v kapitole 3.

Curriculum vitae